

Diario de Decisiones Técnicas e IA — SIGA-UTN

Entrada — 10 de agosto de 2026

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: Integración del módulo de Solicitudes Estudiantiles (ES-01–ES-04) al proyecto SIGA-UTN

Herramienta consultada: Claude (Anthropic), vía chat con acceso a archivos del proyecto

1. Qué se le consultó a la IA

Se le pidió que analizara dos archivos comprimidos:

- `utn-student-requests-module.zip`: un prototipo aislado del módulo de Solicitudes Estudiantiles (portal, motor de reglas, bandeja centralizada), construido a partir de los requerimientos FR-ES-01 a FR-ES-04 del SRS v1.2.
- `SIGA-UTN.zip`: el proyecto real del equipo, con su propia arquitectura ya establecida (contexto `IdentityAccess` bajo DDD granular en `src/`).

La instrucción fue: `"analiza ambos .zip y copia todo lo correspondiente al módulo de Solicitudes Estudiantiles al proyecto de SIGA-UTN."`

2. Qué encontró la IA antes de actuar (y por qué no copió directo)

Antes de mover un solo archivo, la IA reportó tres problemas que una copia mecánica habría ocultado hasta que fallara en `php artisan migrate` o en producción:

1. Dependencias faltantes: las tablas de Solicitudes (`solicitudes`, `reglas_levantamiento`, etc.) dependen por FK de tablas de otros módulos (`courses`, `students`, `periods_academicos`, `historial_academico`) que no existen en SIGA-UTN, porque ese proyecto solo tiene construido el contexto `IdentityAccess` hasta ahora.
2. Convención de nombres inconsistente: el prototipo usa nombres de tabla/columna en español (`solicitudes`, `curso_id`, `estudiante_id`), mientras que todo lo ya construido en SIGA-UTN usa inglés (`permissions`, `roles`, `module`).
3. Arquitectura distinta: SIGA-UTN organiza el código por contexto acotado (`src/``<BoundedContext>``</BoundedContext>``<Aggregate>``</Aggregate>``{Domain,Application,Infrastructure,Presentation}`), mientras que el prototipo usa capas planas (`app/Domain`, `app/Application`, `app/Infrastructure`) — y, según su propio README, solo la capa

Infraestructure está realmente construida; Domain y Application son carpetas vacías con un README.

Esto se registra porque es el punto central del ejercicio: la IA no ejecutó la instrucción literal ("copia todo") sin cuestionarla. Detuvo el trabajo y presentó las tres decisiones al equipo antes de generar una sola línea de código.

3. Qué se aceptó de la respuesta de la IA

- El diagnóstico de las tres inconsistencias (dependencias, idioma, arquitectura) se aceptó íntegramente: coincide con lo que el equipo hubiera encontrado al intentar correr las migraciones.
- La propuesta de traducir todo a inglés para no introducir una mezcla de idiomas en el esquema, dado que ya existe una convención establecida en el proyecto real.
- La propuesta de copiar también las tablas prerequisite (careers, academic_periods, study_plans, levels, courses, course_level, students, student_study_plan, academic_records), aunque no pertenecen estrictamente al alcance de Solicitudes, porque sin ellas el módulo no es funcional.
- El código generado para las 9 migraciones nuevas, 12 modelos Eloquent, 12 factories y 2 seeders, tras revisión manual de nombres de tabla, tipos de columna y relaciones contra la migración y los modelos originales del prototipo.

4. Qué se rechazó y por qué

- Se rechazó adaptar ya mismo la estructura al patrón DDD de SIGA-UTN (src/Requests/... con Domain/Application/Infrastructure/Presentation completos). Razón: el prototipo no tiene esas capas construidas — hacerlo habría significado que la IA *inventara* reglas de negocio, casos de uso y componentes Livewire desde cero, sin que existiera una versión de referencia que validar. El equipo prefirió copiar únicamente lo que existe (Infrastructure) y diseñar Domain/Application propios en una iteración posterior, con criterio del equipo y no generado de cero por la IA.
- Se rechazó copiar los modelos `Rbac/Models` (Permission/Role) del prototipo. SIGA-UTN ya tiene su propia implementación de roles y permisos, tanto en App\Models\Role/Permission como en el contexto DDD src/IdentityAccess. Copiar la versión del prototipo habría creado un modelo paralelo y conflictivo para la misma responsabilidad.
- Se rechazó copiar `DomainServiceProvider.php` del prototipo: es un *stub* vacío pensado para registrar bindings de interfaces de Domain que, como se

explicó arriba, no existen. Incluirlo no aportaba nada funcional y hubiera quedado como código muerto.

5. Qué hubo que corregir o verificar manualmente

- La IA no tuvo acceso a un intérprete de PHP en este entorno, por lo que no pudo ejecutar ``php artisan migrate`` ni las pruebas para validar el resultado. El equipo debe correr las migraciones en un entorno local/CI antes de dar esto por cerrado; la IA lo señaló explícitamente en vez de dar por buena la integración solo por el chequeo estático que sí hizo (balance de llaves, búsqueda de residuos en español).
- Se verificó manualmente que ningún nombre de tabla nuevo (requests, students, courses, etc.) colisionara con nombres reservados o con algo ya existente en el proyecto — la IA no tenía forma de comprobar esto contra una base de datos real.
- Quedó pendiente de decisión del equipo (no resuelto por la IA) el campo `academic_records.equivalence_id`, que no tiene FK real porque depende de un futuro módulo de Repositorio Curricular fuera de alcance. La IA lo dejó documentado en un comentario dentro de la migración en lugar de inventar una tabla equivalences que no fue solicitada.

6. Qué se aprendió del proceso

- Pedir a una IA "copia todo el módulo X" sin especificar destino/convenciones es una instrucción ambigua que puede producir una integración que compila, pero no es coherente con el resto del proyecto.
- Un prototipo aislado (el .zip del módulo de Solicitudes) puede tener piezas construidas de forma desigual (solo Infrastructure, sin Domain/Application). Es responsabilidad del equipo revisar el estado real de lo que se va a integrar y no asumir que "el módulo está completo" solo porque el repositorio existe.
- El valor de usar la IA aquí no fue "que escriba el código", sino que detectara incompatibilidades estructurales antes de generar nada y las presentara como decisiones explícitas del equipo. El criterio de qué convención usar, qué copiar y qué NO inventar lo definió el equipo en cada una de las tres preguntas planteadas — la IA ejecutó una decisión ya tomada, no la tomó por su cuenta.
- Falta correr la suite de migraciones y tests en un entorno real (Docker/CI del proyecto) antes de mergearear esta rama: la revisión de la IA fue estática (sintaxis, nombres, dependencias declaradas), no una prueba de ejecución real.

Entrada — 13 de agosto de 2026

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Definición y creación de 3 perfiles de prueba (Superadmin, Estudiante, Coordinadora de Docencia) para validar el módulo de Solicitudes Estudiantiles

Herramienta consultada: Claude (Anthropic), vía chat para el análisis/diseño y vía Claude Code en terminal para la implementación

1. Qué se le consultó a la IA

En un primer chat se le pidió analizar el repositorio base del docente y compararlo contra el proyecto propio para saber si ya traía usuarios/roles de prueba reutilizables, y luego proponer qué perfiles de prueba crear para poder ver la aplicación "como estudiante" y "como docente/revisor", basándose en:

- El documento oficial del proyecto (Proyecto_4_Solicitudes_Estudiantiles (3).docx).
- El script SQL oficial del sistema completo (sistema_gestion_academica_utn (1).sql), en particular su tabla roles.

Después de acordar los 3 perfiles y sus permisos, se le pidió a la IA un prompt autocontenido para ejecutar la implementación en una segunda sesión (Claude Code en terminal), y finalmente que revisara el resultado antes del commit.

2. Qué encontró la IA antes de actuar (y por qué no aceptó la primera idea del equipo tal cual)

1. El repo del docente no aporta roles académicos. Se verificó archivo por archivo (RoleSeeder.php, PermissionSeeder.php, migraciones) que el proyecto del docente solo siembra Superadmin/Admin — sin Estudiante ni ningún rol de negocio. La idea inicial de "asimilar perfiles ya existentes en la BD del docente" no tenía nada que asimilar; había que crearlos desde cero.

2. La propuesta original del equipo era "Estudiante, Docente, Superadmin". La IA señaló que la palabra "Docente" no aparece ni una sola vez en el documento oficial del proyecto como actor del sistema, y que en el catálogo oficial de roles (sistema_gestion_academica_utn (1).sql) el rol Docente existe, pero representa a un profesor que consulta su oferta de cursos (oferta.consultar, archivos.descargar) — no tiene ningún permiso relacionado con solicitudes. Usar ese rol para "ver la bandeja de revisión" habría dado un usuario de prueba que ni siquiera puede entrar al módulo. El rol correcto, según el propio SRS (ES-01 y ES-04) y la columna solicitudes.revisor_id del esquema oficial (comentario literal: `""Usuario revisor (Docencia/Comisión)""`), es Coordinadora de Docencia.

3. `Superadmin` no sirve para validar criterios de aceptación. La IA explicó que Superadmin tiene un Gate::before que bypassa toda autorización — usarlo como perfil de prueba principal ocultaría cualquier bug real de permisos/scoping, en vez de confirmar que el módulo funciona para un usuario con permisos reales y limitados.

4. Gap real detectado en el código, no solo de nomenclatura:

RequestPolicy::view()/viewAny() no filtran por dueño de la solicitud. La IA lo marcó explícitamente antes de crear el rol Estudiante: con los permisos propuestos, un estudiante vería las solicitudes de **todos** los estudiantes, lo cual contradice el criterio de aceptación literal de ES-03 ("el estudiante debe poder consultar el estado de sus solicitudes"). Se decidió, como equipo, crear los perfiles primero y dejar ese fix como tarea aparte explícita, en vez de que la IA lo resolviera sin que se le pidiera.

3. Qué se aceptó de la respuesta de la IA

- El mapeo final de roles: Estudiante → rol Estudiante (permisos requests.create, requests.view); Docencia → rol Coordinadora de Docencia (permisos requests.view/search/review/export_pdf/export_excel + waiver_rules.create/view/edit/delete); Superadmin se mantiene como está.
- Las credenciales de ejemplo propuestas (estudiante@gmail.com / docencia@gmail.com, clave 12345678), siguiendo el mismo patrón que los usuarios ya existentes en DatabaseSeeder.php.
- Vincular el usuario Estudiante a un registro real de StudentModel (vía user_id), para que exista un expediente contra el cual el módulo pueda operar.
- El prompt autocontenido generado para ejecutar el cambio en una sesión de Claude Code aparte, delegando la escritura del código (decisión explícita del equipo, no una desviación no autorizada).
- El diagnóstico de que no hacía falta ninguna migración nueva: las columnas necesarias (roles.name, students.user_id, requests.user_id) ya existían en el esquema.

4. Qué se rechazó y por qué

- Se rechazó el nombre "Docente" para el rol revisor, por las razones del punto 2: no es un actor del SRS y, en el propio catálogo oficial, ese nombre está reservado para un perfil sin relación con solicitudes.
- Se rechazó usar `Superadmin` como perfil principal de prueba funcional, por el mismo motivo del punto 3 del apartado anterior.
- Se rechazó (por ahora) que la IA implementara el fix de scoping por dueño en `RequestPolicy` junto con la creación de los perfiles, para no mezclar dos

cambios de alcance distinto en el mismo commit — queda como tarea explícita pendiente, documentada aquí.

- Se rechazó el código entregado por la sesión de Claude Code en su primera versión.

5. Qué hubo que corregir o verificar manualmente — el error real de la IA

Al revisar el diff generado por la sesión de Claude Code que implementó los seeders, se encontró que la IA no respetó la convención de nombres ya establecida en el proyecto (identificadores de código en inglés, valores de datos en español): las variables nuevas quedaron en español (\$estudiante, \$docencia, \$estudianteRole, \$estudianteUser, \$docenciaRole, \$docenciaUser) en RoleSeeder.php y DatabaseSeeder.php, a pesar de que el propio código que la IA tenía como contexto (y que ella misma tradujo en la entrada del 10 de agosto) usa inglés de forma consistente en todo lo demás.

Esto es un ejemplo concreto de error de la IA: tuvo el contexto correcto disponible (el resto del archivo, escrito en inglés) y aun así generó variables nuevas en español, probablemente porque copió el idioma de los *valores* de rol ('Estudiante', 'Coordinadora de Docencia') hacia los *nombres de variable*, sin distinguir que esa regla del proyecto aplica solo al contenido de datos, no a los identificadores. Se corrigió con un segundo prompt específico, dando la lista exacta de renombres (\$estudiante → \$studentRole, \$docencia → \$teachingCoordinatorRole, etc.) sin tocar ningún string literal ni la lógica.

Después de la corrección, se verificó:

- Con git diff --stat que solo se modificaron los 2 archivos esperados.
- Contra la base de datos real (vía php artisan tinker) que los 4 roles (Superadmin, Admin, Estudiante, Coordinadora de Docencia) y sus permisos quedaron sincronizados exactamente como se pidió, que los 4 usuarios existen con el rol correcto, y que el StudentModel del usuario Estudiante quedó vinculado (user_id correcto).
- Con php artisan test que la suite sigue en el mismo estado que antes del cambio (32/33, con la única falla preexistente y no relacionada de AuthenticationTest::test_login_screen_can_be_rendered).

6. Qué se aprendió del proceso

- Un nombre de rol "parece correcto" a simple vista si suena razonable (Docente para quien revisa solicitudes de docencia), pero solo verificarlo contra el documento oficial del SRS y el esquema de base de datos reveló que era el rol equivocado. Vale la pena, como equipo, contrastar cualquier

nombre de rol/actor contra la fuente oficial antes de codificarlo, en vez de asumir por el nombre que suena bien.

- Un perfil con bypass total (Superadmin) es útil para administración, pero inútil (y hasta contraproducente) para validar que las reglas de autorización de un módulo funcionan de verdad — probar "como superadmin" puede dar una falsa sensación de que todo funciona.
- La convención "código en inglés, datos en español" no es intuitiva para una IA si el prompt solo le da los nombres de rol en español: hay que ser explícito en el prompt sobre qué parte del código debe traducirse y cuál no, en vez de asumir que la IA va a inferir la regla del contexto general del archivo. Esto quedó confirmado como un error real y verificable (no hipotético) de la IA en este proyecto.
- Delegar la implementación a una sesión de terminal aparte (con un prompt autocontenido y explícito) y luego revisar el diff como paso separado permitió detectar este error antes del commit — revisar el resultado, no solo confiar en que "corrió sin errores", sigue siendo responsabilidad del equipo.
- Queda pendiente, documentado explícitamente para no perderlo: el scoping por dueño en RequestPolicy (para que Estudiante solo vea sus propias solicitudes) y una vista dedicada "Mis Solicitudes" — ninguno de los dos se implementó en este ciclo a propósito.

Entrada — 14 de agosto de 2026

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Diagnóstico y corrección de un bug de legibilidad en los `<select>` del portal de estudiante en modo oscuro

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso al navegador (Chrome) para reproducir el bug visualmente

1. Qué se le consultó a la IA

Se reportó, de forma informal, un bug observado manualmente en el portal de estudiante (/mis-solicitudes) con el tema oscuro activado: al abrir cualquiera de los `<select>` del formulario de nueva solicitud ("Curso a matricular", "Requisito no cumplido", "Curso interno equivalente"), la lista de opciones se veía casi en blanco sobre blanco, ilegible. Se le pidió a la IA que reprodujera el bug y propusiera cómo solucionarlo.

2. Qué encontró la IA antes de actuar — incluyendo un diagnóstico inicial incorrecto

1. La IA reprodujo el bug en un navegador real controlado por Claude Code (login como Estudiante, abrir el formulario, abrir el `<select>`, capturar pantalla) antes de tocar una sola línea de código.
2. Primer diagnóstico (incorrecto): revisando `resources/css/app.css`, notó que el bloque `.form-field` estiliza `label`, `input[type="text"]`, `textarea` e `input[type="file"]`, pero no tiene ninguna regla para `select`. Concluyó que la causa era la ausencia de la propiedad CSS `color-scheme` en `:root/[data-theme="dark"]`, que le indica al navegador si debe dibujar los controles nativos (incluida la lista desplegable de un `<select>`) en variante clara u oscura. Propuso agregar `color-scheme: light / color-scheme: dark` a esos dos bloques.
3. Con autorización explícita del equipo aplicó ese cambio y volvió a probar en el navegador. El bug seguía exactamente igual — el desplegable seguía blanco sobre blanco.
4. En vez de dar el cambio por bueno solo porque "sonaba correcto", la IA verificó con JavaScript en la página real (`getComputedStyle`) que `color-scheme: dark` sí se estaba aplicando correctamente al `<select>` (confirmado: `colorScheme: "dark"`), lo que descartó su propia primera hipótesis — el problema no era que faltara `color-scheme`.
5. Comparó el `<select>` roto contra otro que sí funciona en el resto del sistema (`.control-group select`, usado en los filtros de las pantallas CRUD) y encontró la

diferencia real: `.control-group select` sí define `background: var(--cardBg)` explícito, mientras que `.form-field select` no define ningún fondo y queda transparente. Con fondo transparente, Chromium no pinta nada detrás de las opciones del desplegable nativo, así que se ve el blanco por defecto del sistema operativo con el texto claro (heredado de `--textPrimary` en modo oscuro) casi invisible encima.

3. Qué se aceptó de la respuesta de la IA

- El diagnóstico final (fondo transparente en `.form-field select`, no ausencia de `color-scheme`), una vez verificado en el navegador con capturas de pantalla y `getComputedStyle`.
- La corrección aplicada: agregar `select` a la regla existente `.form-field input[type="text"], .form-field textarea` (mismo `background: var(--cardBg)`, `color: var(--textPrimary)`) y una regla explícita `.form-field select option { background: var(--cardBg); color: var(--textPrimary); }`.
- Mantener también el cambio de `color-scheme` (aunque no fue la causa raíz), por ser buena práctica general para otros controles nativos (scrollbars, checkboxes) que si dependen de esa propiedad.
- La verificación visual final: se probaron los tres `<select>` del portal (levantamiento y convalidación) en modo oscuro, con capturas de pantalla mostrando texto blanco legible sobre fondo azul oscuro en cada opción.

4. Qué se rechazó y por qué

- Se rechazó dar por cerrado el bug después del primer cambio (`color-scheme`) solo porque era la explicación más "de manual" el equipo esperó a que la IA repitiera la prueba visual en el navegador antes de aceptar la corrección, y esa misma prueba fue la que reveló que el primer intento no había funcionado.

5. Qué hubo que corregir o verificar manualmente — el error real de la IA

Este es el caso concreto de error de la IA para esta entrada: el primer diagnóstico y su corrección (`color-scheme`) fueron incorrectos. La IA razonó por analogía con la causa más conocida/documentada de este tipo de bug (falta de `color-scheme` para forzar controles nativos oscuros) sin antes comparar el `<select>` roto contra un `<select>` que sí funciona dentro del mismo proyecto. Ese paso de comparación que hubiera sido más rápido y habría llevado directo a la causa real se hizo únicamente *después* de que la primera corrección fallara la prueba visual, no antes.

Se verificó, en ambos intentos, con:

- Capturas de pantalla del `<select>` abierto, antes y después de cada cambio.
- `getComputedStyle` ejecutado en vivo sobre la página para confirmar qué propiedades CSS se estaban aplicando realmente (`color-scheme`, `color`, `background-color`) en el `<select>` y en sus `<option>`, en lugar de asumir el resultado a partir del CSS fuente.

6. Qué se aprendió del proceso

- Un diagnóstico "razonable" o "de manual" no reemplaza la verificación empírica: la única forma de confirmar que una corrección de CSS realmente funciona es probarla visualmente en el navegador contra el bug original, no solo revisar que el código "se ve correcto".
- Comparar el elemento roto contra un elemento equivalente que sí funciona en el mismo proyecto (`.control-group select` vs. `.form-field select`) fue más eficaz para encontrar la causa raíz que razonar desde documentación general sobre CSS.
- `color-scheme` en CSS afecta la apariencia por defecto de controles nativos, pero no sustituye un `background-color` explícito cuando el elemento (o sus ancestros) deja el fondo transparente: en Chromium, un `<select>` con fondo transparente no hereda un fondo oscuro solo por `color-scheme: dark`, porque el navegador sigue mostrando su lienzo nativo (blanco) detrás del texto heredado.
- Volver a probar en el navegador después de cada cambio, en vez de encadenar varias correcciones antes de verificar, permitió aislar rápidamente que el primer cambio no había sido el culpable real.

Entrada — 14 de agosto de 2026 (continuación)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Validación en tiempo real de archivos adjuntos, subida por arrastrar-y-soltar (drag-and-drop) y botón para quitar un archivo mal adjuntado, en el portal de estudiante

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso al navegador (Chrome) para reproducir y verificar cada cambio

1. Qué se le consultó a la IA

En la misma sesión de trabajo, se reportaron dos bugs adicionales observados manualmente en /mis-solicitudes:

1. Al adjuntar un archivo superior a 5MB en "Nueva solicitud de levantamiento" o "Nueva solicitud de convalidación", el sistema lo deja seleccionar sin avisar nada; el error debería aparecer justo debajo del botón "Seleccionar archivo".
2. En la solicitud de convalidación (3 documentos obligatorios), al enviar sin archivos aparecen correctamente los 3 errores "obligatorio"; pero al adjuntar los archivos uno por uno para corregirlo, el error rojo del campo ya adjuntado se queda ahí en vez de quitarse.

Se le pidió a la IA que primero probara ambos bugs en el navegador para confirmar el problema antes de proponer nada. Después, ya con ambos corregidos, se le consultó qué se podía hacer para agregar arrastrar-y-soltar (drag-and-drop) "como en los sistemas modernos", y luego un botón pequeño de "x" para quitar un archivo mal adjuntado sin tener que recargar la página.

2. Qué encontró la IA antes de actuar

1. Reprodujo ambos bugs en el navegador antes de tocar código: subió un PDF de 6MB sintético (generado con PowerShell) al campo de levantamiento y confirmó que no aparecía ningún error hasta hacer clic en "Enviar solicitud" el backend sí rechazaba el archivo (max:5120 ya existía en WaiverRequestForm/ValidationRequestForm), pero solo al final del flujo, no al seleccionar el archivo. Luego reprodujo el segundo bug en convalidación: adjuntó un PDF válido en "Programa del curso externo" después de un envío fallido y confirmó que el mensaje "es obligatorio" seguía visible pese a que el archivo ya estaba adjunto.

2. Diagnosticó que ambos bugs comparten la misma causa: Livewire no revalida un campo de archivo cuando el usuario lo cambia, solo cuando se llama validate() en el envío completo del formulario. La corrección fue un hook

updated(string \$property) en StudentRequestComponent que llama a \$this->validateOnly(\$property) en cuanto cambia cualquiera de los 4 campos de archivo Livewire detecta automáticamente que la propiedad pertenece a un Form Object y delega la validación a las reglas ya existentes ahí, sin duplicarlas.

3. Verificó, leyendo el código fuente de Livewire

(HandlesValidation::validateOnly), que al validar un campo con éxito el framework limpia automáticamente (resetErrorBag) solo el error de ese campo específico — confirmando, antes de escribir nada, que este único hook resolvía los dos bugs a la vez (error inmediato + limpieza automática del error viejo).

4. Para el drag-and-drop, explicó la opción elegida (envolver el <input type="file"> existente en una zona con Alpine.js, ya incluido vía Livewire) frente a la alternativa de usar la API \$wire.upload() de Livewire directamente, señalando que esta última duplicaría la validación que ya vive en los Form Objects.

5. Para el botón de quitar archivo, identificó que un <input type="file"> nativo no se puede "limpiar" con una X propia por CSS/JS simple — la solución es ocultar el input y mostrar una "pastilla" (nombre + botón ×) cuando el archivo es válido, y que el botón llame a un método Livewire que ponga la propiedad en null.

3. Qué se aceptó de la respuesta de la IA

- El hook updated() con validateOnly() para las 4 validaciones en tiempo real, y el mensaje verde "Archivo adjuntado" cuando el campo queda válido.
- El patrón de drag-and-drop reutilizando el mismo <input> y wire:model ya existentes (vía x-ref + DataTransfer + dispatchEvent(new Event('change'))), en vez de un uploader paralelo.
- El método genérico removeFile(string \$field) (con una lista blanca FILE_FIELDS, la misma ya usada por el hook updated()) en vez de 4 métodos repetidos, uno por campo.
- El patrón de "pastilla" (.file-chip) con el nombre del archivo y un botón × reutilizando los colores de "eliminar" (--actionDeleteBg/--actionDeleteText) ya existentes en el sistema, en vez de introducir una paleta nueva.

4. Qué se rechazó y por qué

- Se rechazó (implícitamente, al no pedirlo) usar \$wire.upload() para el drag-and-drop: habría significado mantener dos caminos de subida y validación en paralelo para el mismo campo.
- Se rechazó dejar el hook updated() genérico sin una lista blanca de campos: la IA lo acotó a los 4 campos de archivo conocidos (FILE_FIELDS) en vez de revalidar cualquier propiedad que cambie, para no disparar validaciones

innecesarias en otros campos del formulario (curso, institución, etc.) que no las necesitan.

5. Qué hubo que corregir o verificar manualmente — el error real de la IA

Después de implementar el drag-and-drop, el equipo notó un efecto visual roto: el texto nativo del navegador ("Ningún archivo seleccionado") aparecía cortado con puntos suspensivos ("Ningún archi...eleccionado"). Este es un bug que la propia IA introdujo al envolver el `<input type="file">` en el nuevo `<div class="dropzone">`.

La causa: `.form-field` es `display:flex; flex-direction:column`, y antes del cambio el `<input>` era hijo **directo** de ese contenedor, por lo que el `align-items: stretch` por defecto de flexbox lo estiraba al ancho completo (~520px), dándole al navegador espacio de sobra para mostrar el texto completo. Al envolverlo en `.dropzone` (un div normal, no flex), el input dejó de ser hijo directo del flex y volvió a su ancho nativo/intrínseco, mucho más angosto — y Chrome trunca con "..." el texto de su propio control nativo cuando no le entra. La IA no anticipó este efecto colateral de layout al proponer el wrapper; lo detectó el equipo visualmente, no la IA de forma proactiva. Se corrigió agregando `.dropzone input[type="file"] {display:block; width:100%; }` para restaurar el mismo ancho de antes.

También se verificó, dado que las herramientas de automatización de navegador no pueden simular un arrastre real de archivos desde el sistema operativo, el flujo de drop completo mediante un archivo sintético (`new File(...)` + `DataTransfer` + evento drop disparado por JavaScript) en ambas pestañas y en el botón de quitar — confirmando visualmente en cada caso el estado esperado (pastilla verde, error limpiado, vuelta a la zona de arrastre tras quitar).

6. Qué se aprendió del proceso

- Un solo hook de validación en tiempo real (`updated()` + `validateOnly()`) resolvió dos bugs reportados por separado porque compartían la misma causa raíz — vale la pena, antes de corregir síntomas por separado, confirmar si comparten un origen común.
- Envolver un elemento nativo (`<input type="file">`) en un contenedor nuevo puede romper silenciosamente el layout que dependía de que ese elemento fuera hijo **directo** de un contenedor flex — este tipo de regresión no se detecta leyendo el código, solo probando visualmente después de cada cambio de estructura HTML, como ya se había aprendido en la entrada anterior del mismo día.
- Cuando una herramienta de prueba no puede replicar una interacción real del usuario (arrastrar un archivo del sistema operativo), simular el evento del navegador con datos sintéticos (`DataTransfer`, eventos `drop/change`)

disparados por JS) es una alternativa válida para verificar la lógica de la aplicación, siempre que quede documentado que no prueba la interacción de bajo nivel del sistema operativo, solo la reacción de la aplicación al evento.

Entrada — 14 de agosto de 2026 (motor de reglas y fecha estimada)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Conexión del motor de reglas de ES-01 (waiver engine), detección de duplicados, y la fecha estimada de resolución de ES-03 (entrada manual + asignación automática tras 24h)

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso al navegador para probar en vivo con los 3 perfiles (Estudiante, Docencia, Superadmin)

1. Qué se le consultó a la IA

El docente pidió que todos los CRUDs del proyecto estuvieran listos para una revisión al día siguiente (15 de agosto). Se le pidió a la IA verificar si el CRUD del perfil Estudiante estaba completo. Al confirmar que faltaba el motor de reglas de ES-01, se le pidió conectarlo, con una aclaración explícita de negocio: toda solicitud enviada debe quedar en estado "Pendiente" hasta que Docencia la revise y decida no debía auto-resolverse aunque el motor concluyera un resultado. Después se le pidió resolver también la fecha estimada de resolución de ES-03, dejando explícitamente fuera del alcance de esta sesión la notificación por correo electrónico (esa se dejó documentada para un tercer avance posterior del curso, no para la entrega de mañana).

2. Qué encontró la IA antes de actuar

1. Antes de tocar código, releyó el documento oficial del proyecto completo (extraído manualmente del .docx con unzip + sed, ya que no hay Python instalado en este entorno) para confirmar los criterios de aceptación exactos de ES-01, ES-03 y ES-04, en vez de trabajar de memoria.
2. Verificó, probando en el navegador como Estudiante, que el motor de reglas no evaluaba nada: toda solicitud quedaba "Pendiente de revisión" sin importar las reglas configuradas — confirmado además por un comentario explícito ya existente en CreateRequestUseCase.php ("Running the waiver/validation engine... is intentionally NOT done here").
3. Encontró que la infraestructura de datos para el motor ya existía sin usarse: la entidad Request ya tenía los campos engineResult/violatedRuleId, y el repositorio ya los persistía solo faltaba la lógica de evaluación y quién la llamara.
4. Ante la ambigüedad del spec sobre si el motor debía cerrar la solicitud automáticamente o no, la IA no asumió una respuesta: preguntó directamente al equipo. Al recibir la regla de negocio (todo queda Pendiente hasta que Docencia decida), señaló que esa decisión concilia mejor ES-01 con ES-04 (que exige que

toda solicitud, no solo las no concluyentes, aparezca en la bandeja de Docencia) algo que no era obvio a primera lectura de ES-01 en aislamiento.

5. Para la fecha estimada, revisó routes/console.php (sin comandos programados) y el modal de revisión existente (sin campo de fecha), confirmando que ninguna de las dos partes del criterio de ES-03 estaba implementada.

3. Qué se aceptó de la respuesta de la IA

- El diseño del motor (WaiverEngine como Domain Service puro + StudentAcademicProfileRepositoryInterface como puerto + adaptador Eloquent), con la decisión documentada de que la primera regla *activa* en orden es la autoritativa (evita ambigüedad sobre qué pasa con reglas subsecuentes).
- La detección de duplicados antes de correr el motor, con el mensaje exacto del spec ("Este levantamiento ya fue procesado previamente").
- Que status nunca se autorresuelva — el engineResult queda como sugerencia visible para Docencia, no como una acción automática.
- Aplicar la asignación automática de fecha estimada de forma perezosa (al leer una solicitud), no vía un job programado — así funciona de forma confiable en la demo sin depender de que un scheduler esté corriendo.
- Reutilizar el modal de revisión ya existente para que el revisor ingrese la fecha, en vez de construir una pantalla nueva.

4. Qué se rechazó y por qué

- Se rechazó implementar la notificación por correo de ES-03 en esta sesión. El equipo aclaró explícitamente que esa funcionalidad corresponde a un tercer avance posterior del curso, no a la entrega de mañana — aunque la IA ya había investigado el gap (confirmó que no existe ningún Mail::Notification::Mailable en el proyecto) y lo tenía listo para proponer, se frenó la implementación a pedido del equipo.
- Se rechazó que la IA decidiera por su cuenta el comportamiento de auto-resolución del motor se le pidió la regla de negocio explícitamente en vez de dejarla inferir del spec.

5. Qué hubo que corregir o verificar manualmente — el error real de la IA

Al escribir por primera vez las fixtures de prueba del motor en TestDataSeeder.php (curso + reglas + expediente académico del estudiante demo), la IA asumió que ese seeder corría después de que DatabaseSeeder creara el usuario estudiante@gmail.com. En realidad corría antes: TestDataSeeder estaba en el bloque \$this->call([...]) inicial, y los 4 usuarios de prueba (incluido estudiante@gmail.com) se creaban más abajo, fuera de ese

bloque. El código tenía un guard silencioso (`if ($demoUser === null) { return; }`) que hizo que las fixtures del motor se saltaran sin ningún error visible en la consola — `migrate:fresh --seed` terminó con "DONE" en todos los seeders, dando la falsa impresión de que todo se había sembrado correctamente.

El error se detectó únicamente porque, en vez de confiar en la salida de la consola, se verificó directamente contra la base de datos (`DB::table('waiver_rules')->count()` devolvió 0 cuando debía ser 2). Se corrigió moviendo la llamada a `TestDataSeeder` al final de `DatabaseSeeder::run()`, después de crear los 4 usuarios.

También se detectó, verificando el código existente antes de dar por completa la nueva funcionalidad, que `EloquentRequestRepository::save()` nunca escribía la columna `estimated_resolution_date` un bug preexistente (no introducido en esta sesión) que habría hecho que cualquier fecha ingresada por el revisor se perdiera silenciosamente al guardar. Se corrigió como parte del mismo cambio.

Se verificó todo con datos reales, no solo lectura de código:

- Los 3 resultados del motor (Aprobado/Denegado/Revisión manual), probados en vivo como Estudiante contra un expediente académico sembrado a propósito con notas específicas.
- El flujo de duplicados de punta a punta: aprobar una solicitud como Docencia, reintentar el mismo levantamiento como Estudiante, y confirmar tanto el mensaje en pantalla como que no se creó una fila nueva en requests.
- El cálculo de "5 días hábiles" contra un calendario real (jueves 13 de agosto + 5 días hábiles, saltando el fin de semana, da jueves 20 de agosto) no se asumió que la lógica de fechas estuviera bien solo porque compilaba.

6. Qué se aprendió del proceso

- Un comentario de diseño ya escrito en el código ("intentionally NOT done here") es más confiable que inferir del nombre de una clase si algo está implementado — vale la pena leerlos antes de dar por completo un requerimiento.
- Un guard silencioso en un seeder (`if (...) return;`) es peligroso si el resultado no se verifica contra la base de datos real: la consola puede reportar éxito, aunque la lógica interna nunca haya corrido.
- Frente a una regla de negocio ambigua en el spec (¿el motor cierra la solicitud o no?), preguntar directamente al equipo evitó construir algo que hubiera que rehacer y en este caso la respuesta del equipo terminó siendo la que mejor concilia dos requerimientos (ES-01 y ES-04) que a primera vista parecían apuntar en direcciones distintas.

- Aplicar una corrección de forma perezosa (al leer, no por un job programado) es una estrategia más segura para una demo en vivo cuando no se puede garantizar que un proceso en segundo plano esté corriendo.
- Documentar explícitamente qué se deja fuera de una sesión a propósito (como la notificación por correo, pospuesta a un tercer avance) es tan importante como documentar qué se hizo evita que en una sesión futura, o en la defensa oral, se confunda una decisión de alcance con un olvido.

Entrada — 14 de agosto de 2026 (escaneo de CRUDs y permisos de Docencia)
Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Escaneo completo de los CRUD del proyecto por perfil (Estudiante, Docencia, Superadmin/Admin) de cara al avance del 15 de agosto, y corrección de un permiso faltante en RoleSeeder.php

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso al navegador para probar los 3 perfiles en vivo

1. Qué se le consultó a la IA

El equipo pidió, en varios pasos: (1) confirmar que los CRUD de los perfiles Docencia y Superadmin estuvieran listos para el avance de mañana (solo la parte de persistencia/CRUD — se aclaró explícitamente que lo funcional del motor de reglas se deja para otro avance); (2) investigar si el rol Admin y Superadmin, al terminar ambos con acceso total, eran un diseño redundante o un error; (3) verificar esa respuesta contra la plantilla oficial del profesor (C:\SIGA-UTN- RepoProfe\SIGA), no solo contra el manual.

2. Qué encontró la IA antes de actuar

1. Escaneando la estructura completa del proyecto

(src/**/Presentation/Livewire), identificó los 5 CRUD reales del sistema: Roles, Permisos (ambos del starter kit, no del módulo del equipo), Reglas de Levantamiento, Convalidaciones de Cursos y la Bandeja de solicitudes más la pantalla de autoservicio del Estudiante, que es Create+Read por diseño, no un CRUD completo.

2. Revisando RoleSeeder.php, encontró que el rol "Coordinadora de Docencia" no tenía ningún permiso validation_precedents.* el link ni aparecía en su menú y la URL directa daba "Acceso denegado". Lo cruzó contra el documento oficial: ES-04 dice que la bandeja de Docencia agrupa "todas las solicitudes estudiantiles (levantamientos y convalidaciones)", lo que implica que Docencia ya opera el flujo completo de convalidaciones vía revisión (requests.review), pero no podía tocar el catálogo de precedentes que alimenta esa misma revisión.

3. Sobre la pregunta si Admin/Superadmin son redundantes: primero encontró en Manual - Como crear un CRUD nuevo en SIGA-UTN.docx una cita textual que sugería que Admin debía arrancar sin ningún permiso, asignado manualmente después vía UI lo cual, de ser cierto, habría significado que el proyecto se desvió del patrón documentado.

4. En vez de aplicar esa cita directamente, la IA fue a verificarla contra el código real del profesor y encontró que el propio DatabaseSeeder.php del profesor no solo RoleSeeder.php— sincroniza el 100% de los permisos a Admin para la cuenta de prueba admin@gmail.com, exactamente igual que el proyecto del equipo. La cita del manual describe el comportamiento de RoleSeeder.php aislado (útil para quien solo corre ese seeder), no el comportamiento real del sistema completo tras DatabaseSeeder.php.

3. Qué se aceptó de la respuesta de la IA

- El diagnóstico de los 5 CRUD reales del sistema y cuáles corresponden al alcance del equipo (Solicitudes Estudiantiles) versus cuáles son plantilla compartida del profesor (Roles, Permisos, y las demás secciones del sidebar como Oferta académica, Docentes, Aulas, Grupos, Riesgos, Reportes).
- Extender RoleSeeder.php dándole a Docencia los 4 permisos validation_precedents.create/view/edit/delete, en vez de crear un rol nuevo "Comisión Técnica de Convalidaciones" (que el spec nombra literalmente para ES-02 pero que el sistema nunca implementó como rol separado).
- La conclusión final de que Admin/Superadmin no requieren ningún cambio: coinciden exactamente con el repo de referencia del profesor.

4. Qué se rechazó y por qué

- Se rechazó aplicar el cambio sugerido por la primera lectura del manual (hacer que Admin arrancara sin permisos) sin antes contrastarlo contra el código real del profesor el equipo pidió explícitamente "escaneando el proyecto del profe... como lo hizo" en vez de conformarse con la cita de la documentación.
- Se rechazó, en una ronda anterior de esta misma conversación, crear un rol nuevo "Comisión Técnica" evaluado como más fiel a la letra del spec, pero de mayor alcance (rol nuevo, usuario demo nuevo) para un problema que ES-04 ya resuelve dándole el flujo completo a Docencia en una sola bandeja.
- Se rechazó tocar cualquier archivo de los módulos compartidos de la plantilla (Roles, Permisos, DatabaseSeeder.php) una vez confirmado que coinciden con el repo del profesor el equipo fue explícito en que su entrega es solo el módulo de Solicitudes Estudiantiles.

5. Qué hubo que corregir o verificar manualmente

- La cita del manual, tomada de forma aislada, habría llevado a una conclusión incorrecta (que el proyecto tenía una desviación que corregir). Solo comparando línea por línea contra C:\SIGA-UTN-RepoProfe\SIGA\database\seeders\DatabaseSeeder.php se confirmó que no

había ninguna desviación real el manual describe un paso intermedio (RoleSeeder.php solo), no el resultado final del sistema completo.

- Se verificó en vivo, logueado como Docencia, que el permiso nuevo realmente habilitaba el acceso: el link "Convalidaciones de Cursos" pasó de no aparecer en el menú a aparecer con el botón "Agregar" disponible.
- Se verificó, tras crear y eliminar un rol de prueba ("Rol de Prueba QA") durante el escaneo de CRUD, que no quedó ningún dato residual: se confirmó contra la base de datos que persisten exactamente los 4 roles originales, los 36 permisos originales, y cero filas huérfanas en la tabla pivote permission_role.
- Se corrió la suite de tests (32/33, la única falla preexistente y no relacionada) después del cambio en RoleSeeder.php para confirmar que no introdujo regresiones.

6. Qué se aprendió del proceso

- Una cita de documentación, aunque sea del propio profesor, puede describir un paso intermedio del proceso y no el comportamiento final del sistema vale la pena verificar contra el código real (cuando está disponible) antes de aplicar un cambio basado solo en el texto de un manual.
- Separar con claridad qué es "nuestro módulo" de qué es "plantilla compartida" evitó dos horas de posible trabajo innecesario.
- Verificar que una prueba en vivo (crear+eliminar un rol de prueba) no dejó residuos, contrastando contra la base de datos real, es la misma disciplina aplicada en entradas anteriores del diario: no asumir que "no dio error" significa "quedó limpio".

Entrada — 19 de agosto de 2026

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Subida de documentos en el modal "Nueva solicitud" de la bandeja de Docencia (feedback del profesor en el 2do avance) y aclaración del flujo lógico de creación de solicitudes por rol

Participantes: Equipo de desarrollo ISW-521 (un integrante trabajando sobre los puntos 1 y 2 de un plan repartido con su compañero)

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso al navegador (Chrome real del usuario) para probar en vivo

1. Qué se le consultó a la IA

El profesor, en el 2do avance, pidió agregar subida de documentos al modal "Nueva solicitud" de la bandeja de Docencia (RequestComponent), que hasta ese momento no tenía ningún campo de archivo. El equipo repartió el trabajo con su compañero en un plan de 5 puntos (compartido como captura de pantalla) y a esta sesión le correspondieron los puntos 1 y 2: el Form Object (RequestForm.php) y el componente/vista (RequestComponent.php + su blade). Se pidió explícitamente: implementar el cambio 1, explicarlo y confirmarlo antes de seguir con el cambio 2; después probarlo en el navegador con las credenciales reales de cada perfil; y, antes del commit, revisar un documento adicional (la boleta oficial SLR-002 V.3 — Solicitud de Levantamiento de Requisito, compartida por el docente) para confirmar que el cambio no se saliera del dominio.

2. Qué encontró la IA antes de actuar

1. Antes de tocar código, ubicó los 4 archivos exactos que el plan del compañero mencionaba (RequestForm.php, WaiverRequestForm.php, ValidationRequestForm.php, el trait StoresRequestAttachments) y confirmó que el patrón de subida ya estaba resuelto y probado en el portal de estudiante el trabajo real era reutilizarlo en el formulario de Docencia, no diseñarlo de nuevo.

2. Al leer el PDF oficial SLR-002 que el docente compartió como referencia, encontró una nota explícita en el propio documento: "Esta boleta es de uso exclusivo de la dirección de carrera, área o programa" — es decir, las 5 opciones de justificación (a-e) del formulario no son responsabilidad del sistema ni del estudiante/Docencia, es un trámite interno de Registro Universitario que corre en paralelo. Antes de asumir que el sistema necesitaba un campo nuevo para esas 5 opciones, la IA le presentó al equipo la disyuntiva explícita (solo adjuntar el PDF vs. agregar un campo estructurado) en vez de decidir unilateralmente.

3. Al probar el flujo con el usuario real de Docencia (docencia@gmail.com), encontró que el botón "Nueva solicitud"/"Agregar" no aparece para ese rol: el seeder (RoleSeeder.php) nunca le asignó el permiso requests.create a "Coordinadora de Docencia" solo tiene requests.view/search/review/export_pdf/export_excel y los de waiver_rules/validation_precedents. Esto no es un bug introducido por el cambio; es el estado preexistente del sistema, pero cambiaba por completo con qué usuario había que probar la funcionalidad.

4. Antes de decidir con qué perfil probar, releyó ES-01/ES-02/ES-04 del documento oficial del proyecto y confirmó que el spec nunca describe a Docencia como creador de solicitudes el flujo documentado es siempre Estudiante → Portal → Motor → Bandeja de Docencia. Esto confirmó que la ausencia de requests.create en Docencia no es un gap que haya que corregir, sino que es coherente con el spec.

3. Qué se aceptó de la respuesta de la IA

- El diseño del cambio 1: 4 propiedades de archivo + reglas required_if:type,... + reutilización del trait StoresRequestAttachments en RequestForm.php, sin tocar RequestDTO (el parámetro attachments ya existía) ni ninguna capa de Domain/Application.
- El diseño del cambio 2: WithFileUploads + updated()/removeFile() en el componente, y los dropzones (1 para Dispensa, 3 para Convalidación) copiados del mismo patrón ya usado en el portal de estudiante.
- La lectura del PDF SLR-002: que el sistema solo necesita el archivo genérico como adjunto, sin capturar las 5 justificaciones como dato estructurado confirmado por el propio equipo tras leer la nota "de uso exclusivo de la dirección de carrera" del documento.
- Probar con Superadmin (no con Docencia) al confirmarse que Docencia no tiene ni debería tener requests.create según el spec validado end-to-end: creación de una solicitud de Dispensa con archivo real adjunto, verificación de que el archivo quedó listado con enlace de descarga en el detalle, y verificación visual de que los 3 dropzones de Convalidación renderizan al cambiar el tipo. Los datos de prueba se eliminaron después.
- La recomendación final de no agregarle requests.create a Docencia, sustentada en que ES-01/ES-02/ES-04 nunca la describen como creadora de solicitudes evitando un cambio de permisos no pedido que hubiera sido difícil de justificar en la defensa oral.

4. Qué se rechazó y por qué

- Se rechazó (tras la respuesta del equipo a la pregunta directa) modelar las 5 justificaciones del SLR-002 como un campo nuevo en el formulario — el propio documento aclara que es un trámite exclusivo de Dirección de Carrera, no del sistema.
- Se rechazó seguir intentando probar el cambio con el usuario docencia@gmail.com una vez confirmado que el botón no le aparece por diseño (falta de permiso, no un bug del cambio actual) — se cambió a Superadmin para no bloquear la verificación funcional.
- Se rechazó, en la conversación posterior, agregarle requests.create a Docencia "para que el modal le sirva" — el equipo prefirió seguir la letra del spec (Docencia revisa, no crea) antes que hacer que un permiso encajara con un modal que en realidad es una herramienta administrativa general (Admin/Superadmin), no parte de los 4 requerimientos funcionales.

5. Qué hubo que corregir o verificar manualmente

- Al probar en el navegador, el clic sobre "Nueva solicitud"/"Agregar" y sobre "Cerrar sesión" no siempre registró en el primer intento (el modal/menú tardaba en montarse); se verificó cada acción con una captura de pantalla posterior en vez de asumir que el clic había funcionado, y se repitió el clic cuando la captura mostraba que no había pasado nada — evitando reportar un falso positivo de que el modal no abría.
- Se generó un PDF sintético mínimo (cabecera %PDF-1.4 válida) para poder subir un archivo real de prueba, ya que las herramientas de automatización de navegador solo pueden adjuntar archivos a los que la sesión tiene permiso de lectura explícito (no el Downloads del usuario sin autorización previa).
- Tras el envío, no se asumió que "la fila apareció en la tabla" era prueba suficiente: se abrió el modal de detalle y se confirmó, dentro de "Documentos adjuntos", que el archivo específico subido (test-support-document.pdf) quedó listado con su enlace de descarga — confirmando que storeAttachment() y RequestAttachmentDownloadController (ya existentes) siguen funcionando con el nuevo origen de datos.
- Se eliminó explícitamente la solicitud de prueba y el archivo local sintético al terminar, para no dejar datos falsos en la base de datos de la demo.

6. Qué se aprendió del proceso

- Un documento de referencia compartido por el docente (el SLR-002) puede parecer, a primera vista, una fuente de nuevos requisitos de datos pero una nota al pie del propio documento ("uso exclusivo de la dirección de carrera") puede indicar exactamente lo contrario: que ese contenido es contexto de

dominio, no un campo a implementar. Vale la pena leer las notas/condiciones del documento completo, no solo su formulario visible, antes de traducirlo a un cambio de código.

- Que un botón de acción no aparezca para un rol específico no siempre es un bug: cruzarlo contra el documento oficial (¿ese rol debería poder hacer esto según el spec?) evita "arreglar" un permiso que en realidad está bien como está.
- Verificar con qué usuario tiene sentido probar una funcionalidad es una decisión que depende del spec, no solo de con qué usuario "funciona el botón" usar Superadmin para probar aquí fue correcto porque Docencia legítimamente no debe tener este permiso, no porque Superadmin sea la opción más cómoda.
- Confirmar visualmente cada clic con una captura antes de asumir que registró evitó reportar como probado algo que en realidad no había pasado la misma disciplina de "verificar, no asumir" que ya aparece en entradas anteriores de este diario, aplicada ahora a la interacción con la UI en tiempo real y no solo al código o a la base de datos.

Entrada — 20 de agosto de 2026

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Generación de archivos de prueba de subida de documentos para los 4 perfiles (de cara a la revisión informal del docente) y diagnóstico/corrección de un Internal Server Error real encontrado al probar la solicitud de convalidación como Estudiante

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso al navegador para reproducir el error

1. Qué se le consultó a la IA

De cara a mostrarle el proyecto al docente al día siguiente, se le pidió a la IA crear, dentro de C:\ARCHIVO DE PRUEBAS (fuera del repositorio, explícitamente aclarado que no debía tocar el proyecto), una carpeta por cada uno de los 4 perfiles de prueba (Superadmin, Admin, Estudiante, Docencia) con archivos PDF/imagen/otro formato, livianos y pesados, para poder probar en vivo si el sistema acepta o rechaza cada uno al adjuntar documentos.

2. Qué encontró la IA antes de actuar

1. Antes de generar un solo archivo, revisó la regla de validación real en el código (WaiverRequestForm.php, ValidationRequestForm.php, RequestForm.php): mimes:pdf,jpg,jpeg,png, max:5120 (5120 KB = 5 MB, límite inclusivo) en vez de asumir formatos o tamaños de límite a partir del enunciado del usuario.
2. Verificó, buscando en todo src/ qué componentes usan WithFileUploads, que solo los perfiles Estudiante (StudentRequestComponent) y Docencia (RequestComponent, modal "Nueva solicitud" agregado en el avance anterior) tienen formularios con subida de archivos en este módulo Superadmin y Admin no exponen ningún campo de archivo en Solicitudes. En vez de fabricar una prueba de algo que no existe en el código, se lo señaló explícitamente al equipo y se generaron solo 2 archivos de referencia + una nota para esos dos perfiles.
3. Generó los archivos con cabeceras reales válidas (PDF con estructura mínima real, JPG/PNG creados con System.Drawing de .NET) y les agregó *padding* para alcanzar tamaños objetivo exactos, incluyendo una prueba de borde exacto sobre el límite max:5120: un archivo de 5120 KB exactos (debe aceptar) y uno de 5121 KB (debe rechazar por 1 KB de más) para que, si algo falla, se sepa que falla por tamaño y no por un archivo corrupto.

3. Qué se aceptó de la respuesta de la IA

- El set de archivos generado por perfil (Validos/, Limite_5MB/, Invalidos/) y el README.txt con la guía de pruebas paso a paso y las credenciales de los 4 perfiles.
- La decisión de no fabricar archivos de prueba de subida para Superadmin/Admin más allá de una referencia mínima, documentando la razón (esos roles no tienen esa funcionalidad en el módulo) en vez de simular una prueba sin sentido.

4. Qué se rechazó y por qué y el bug real que salió de la prueba

Al probar manualmente (no la IA, el equipo) el envío de una solicitud de convalidación como Estudiante usando los archivos generados, apareció un Internal Server Error real: TypeError: htmlspecialchars(): Argument #1 (\$string) must be of type string, array given, en student-request-component.blade.php:256. Se le pidió a la IA diagnosticarlo.

La IA encontró la causa raíz revisando el código, no adivinando por el mensaje de error: la vista llamaba `__($successType)` con `$successType = 'Validation'` (uno de los dos tipos de solicitud del dominio). Como en Windows el sistema de archivos no distingue mayúsculas/minúsculas, Laravel interpreta la cadena "Validation" (sin punto) como el nombre del grupo de traducción validation, y como sí existe `lang/es/validation.php`, `__()` devuelve el array completo de mensajes de validación en vez de un texto de ahí el array given. No fue un problema con los archivos subidos (los 3 documentos y la fila de la solicitud se guardaron correctamente en la base de datos, según el log de queries del propio error de Laravel); el crash ocurría solo al renderizar el modal de éxito.

Antes de proponer una corrección, la IA comparó contra `request-component.blade.php` (el componente de Docencia) y encontró que ese mismo bug ya había sido evitado ahí, usando un `match()` explícito (`'Validation' => __('Course Validation')`) en vez de pasar el tipo crudo a `__()`. Se rechazó, por dos razones, renombrar el valor de dominio 'Validation' en sí (usado en lógica de negocio, DTOs y la columna `requests.type`): habría sido un cambio de mayor alcance del necesario para arreglar un bug de presentación, y ya existía un patrón correcto y probado en el propio proyecto para replicar. Se aplicó ese mismo `match()` en los 3 lugares de `student-request-component.blade.php` que tenían el problema (tabla "Mis solicitudes", modal de éxito, modal de detalle).

5. Qué hubo que corregir o verificar manualmente

- Se verificó que el mismo patrón vulnerable (`__($valor_dinámico)` sobre un tipo de dominio) no existiera en ningún otro lugar de `request-`

component.blade.php (Docencia): ya usaba match() correctamente en sus 2 puntos de visualización del tipo, por lo que no necesitó cambios.

- El equipo repitió en su propio navegador el mismo envío de convalidación que había fallado, con los mismos archivos, y confirmó que tras la corrección el modal "Request submitted!" se muestra correctamente sin error 500.

6. Qué se aprendió del proceso

- Generar datos de prueba realistas (tamaños exactos en el límite, formatos con cabeceras válidas) no solo prueba la validación esperada — en este caso, el ejercicio de probar en vivo con esos archivos fue lo que hizo aparecer un bug real y no relacionado con el objetivo original (una colisión de nombres con __()), que no se habría detectado solo leyendo el código.
- Un bug de traducción como este depende del sistema operativo: en Windows (sistema de archivos insensible a mayúsculas) __('Validation') colisiona con lang/es/validation.php; en Linux/Mac (sensible a mayúsculas) probablemente no se habría manifestado igual. Esto es relevante para el equipo porque el entorno de desarrollo es Windows, pero el de despliegue/evaluación podría no serlo un bug así puede aparecer o desaparecer según dónde se corra, por lo que no basta con "no me dio error en mi máquina".
- Antes de corregir un bug de traducción/presentación, vale la pena revisar si el mismo patrón ya fue resuelto correctamente en otra parte del propio proyecto (aquí, el componente de Docencia) replicar una solución ya validada en el propio código es más seguro que diseñar una nueva desde cero.
- Pasar un valor de dominio dinámico directamente como clave a __() es frágil en general: cualquier valor que coincida con el nombre de un archivo de idioma reservado de Laravel (auth, validation, passwords, pagination, etc.) puede devolver un array en vez de una traducción. La forma segura, ya usada en este proyecto, es mapear explícitamente los valores de dominio conocidos a textos traducibles fijos con match(), nunca traducir el dato crudo directamente.

Entrada — 20 de agosto de 2026 (continuación — pruebas guiadas por perfil, exportación PDF/Excel y traducciones faltantes)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Sesión de pruebas manuales guiadas por pantalla y por perfil (Docencia y Admin), diagnóstico de un error 500 al exportar PDF/Excel, y corrección de etiquetas del filtro de la bandeja que aparecían en inglés

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso al navegador para verificar en vivo

1. Qué se le consultó a la IA

De cara al avance de mañana, el equipo decidió probar manualmente en cada perfil (Docencia primero, luego Admin) y el objetivo de dominio de cada prueba no que la IA hiciera las pruebas, sino que las explicara para entender mejor el dominio antes de ejecutarlas a mano. Durante ese recorrido surgieron tres problemas reales que sí se le pidió corregir: (1) un error 500 al exportar PDF/Excel desde la bandeja de Docencia, (2) confirmar si "aprobar" un precedente de convalidación debía reflejarse como resultado inmediato en la solicitud del estudiante, y (3) el filtro de la bandeja (Docencia/Admin) con las etiquetas "Program", "Received from" y "Received to" en inglés en medio de una interfaz en español.

2. Qué encontró la IA antes de actuar

1. Exportación PDF/Excel: revisando el stack trace del error, encontró que spatie/laravel-pdf y spatie/simple-excel estaban declarados en composer.json/composer.lock pero nunca se habían instalado (vendor/spatie/ no existía). Al intentar composer install, encontró un segundo problema no relacionado con el proyecto: un bug conocido de Composer al leer curl_version() en Windows cuando el backend SSL es Schannel sin número de versión (el caso exacto del PHP de Herd usado en este equipo) resuelto usando una segunda instalación de PHP presente en la máquina (C:\php\php.exe, con curl/OpenSSL en formato normal) solo para ese paso puntual de instalación. Tras instalar, un tercer problema: Browsershot (usado por spatie/laravel-pdf para generar el PDF) requiere un Chrome headless descargado aparte vía npx puppeteer browsers install chrome-headless-shell, que tampoco se había hecho nunca.

2. Resultado de convalidación en "Mis solicitudes": en vez de asumir que era un bug, revisó CreateRequestUseCase::handle() y confirmó que el motor automático (WaiverEngine) solo corre para "Dispensa de requisito" para "Convalidación" el engineResult siempre queda null a propósito; el precedente aprobado solo se vincula silenciosamente (resolveValidationPrecedentId()) y se

le muestra a Docencia al revisar, nunca al estudiante como resultado inmediato. Confirmó que el comportamiento observado era el diseño esperado, no un defecto.

3. Etiquetas en inglés: en vez de traducir solo las 3 etiquetas señaladas por el equipo, comparó todas las claves __('...') usadas en las vistas del módulo de Solicitudes contra el diccionario lang/es.json (con un script PHP de una sola pasada) y encontró 2 claves adicionales sin traducir que no eran visibles a simple vista en la captura compartida ("All programs", "Clear filters") evitando dejar el arreglo incompleto.

4. Error propio detectado y corregido en la misma sesión: al explicar qué probar como Admin, la IA había dicho antes que "Superadmin y Admin no tienen subida de archivos en este módulo" una generalización incorrecta. Al revisar DatabaseSeeder.php:38-39, encontró que el rol Admin recibe el 100% de los permisos (incluido requests.create), el mismo permiso que le falta a Docencia y que es justamente el que oculta el botón "Nueva solicitud" (con sus 4 campos de archivo) en esa bandeja. La IA señaló su propio error explícitamente al equipo en vez de dejarlo pasar, y corrigió el plan de pruebas de Admin en consecuencia.

3. Qué se aceptó de la respuesta de la IA

- Instalar las dependencias de Composer usando temporalmente C:\php\php.exe (con --ignore-platform-req=ext-fileinfo y habilitando puntualmente extension=zip en ese php.ini aparte, revertido después) — verificado primero que el PHP real de la aplicación (Herd) ya tenía curl, fileinfo y zip cargados, así que no había riesgo de que el runtime real quedara con dependencias a medias.
- La instalación del navegador headless de Puppeteer (chrome-headless-shell) como paso necesario, no opcional, para que Browsershot funcione.
- Las 5 traducciones agregadas a lang/es.json ("Program" → "Programa", "All programs" → "Todos los programas", "Received from" → "Recibida desde", "Received to" → "Recibida hasta", "Clear filters" → "Limpiar filtros"), verificadas en el navegador sin necesidad de reiniciar el servidor.
- La corrección del plan de pruebas de Admin para incluir la subida de archivos en /solicitudes, una vez confirmado el permiso real.

4. Qué se rechazó y por qué

- Se rechazó dejar habilitada permanentemente la extensión zip en C:\php\php.ini: esa instalación de PHP no es la que corre la aplicación, solo se usó como herramienta puntual para esquivar el bug de Composer, así que se revirtió al estado original apenas terminó la instalación.

- Se rechazó tocar cualquier archivo `.blade.php` para arreglar las traducciones el problema estaba en el diccionario (`lang/es.json`), no en las vistas, así que modificar las vistas habría sido un cambio innecesario y hubiera dejado el `.blade.php` inconsistente con el resto de claves ya traducidas por diccionario.

5. Qué hubo que corregir o verificar manualmente — el error real de la IA

El error real de esta sesión ya está descrito en el punto 2.4: la IA afirmó incorrectamente, sin verificarlo contra el código, que Admin no tenía subida de archivos en el módulo. Se corrigió solo porque el equipo pidió explícitamente el plan de pruebas de Admin, lo que obligó a revisar el permiso real antes de dar la guía; si el equipo no hubiera pedido probar ese perfil, el error habría quedado sin detectar en la documentación de la sesión anterior.

Se verificó, en cada uno de los tres arreglos, contra evidencia real y no solo contra el mensaje de éxito de la terminal:

- La exportación PDF y Excel se probó en vivo en el navegador, se confirmó 200 OK en la petición de red para ambos formatos antes de darlo por resuelto, después de que el primer intento de PDF diera 500 por el Chrome headless faltante.
- El resultado de convalidación se verificó leyendo el código de `CreateRequestUseCase`, no solo probando en pantalla, para poder explicar la razón exacta (no solo "así funciona").
- Las traducciones se verificaron extrayendo el texto real de la página (`get_page_text`) tras recargar, no solo revisando que el JSON quedara bien formado.

6. Qué se aprendió del proceso

- Un `composer.json/composer.lock` con una dependencia declarada no garantiza que esté instalada vale la pena, ante una clase "not found", verificar primero si `vendor/<paquete>` existe físicamente antes de sospechar de un bug de código.
- Un entorno Windows con múltiples instalaciones de PHP (Herd para correr la app, otra instalación aparte) puede ser una salida práctica cuando una de ellas tiene un bug de compatibilidad con una herramienta (Composer) — siempre que se verifique antes que el PHP que sí importa (el que corre `php artisan serve`) tenga todo lo necesario, para no arrastrar el problema al runtime real.
- Una librería de generación de PDF basada en un navegador headless (Browsershot/Puppeteer) tiene una dependencia binaria (el propio Chrome) que Composer no instala — es un paso de instalación aparte, fácil de olvidar,

que conviene documentar en el README del proyecto para que no se repita este mismo diagnóstico en otra máquina.

- Diferenciar "esto no hace lo que esperaba" de "esto es un bug" requiere leer el caso de uso real (CreateRequestUseCase) en vez de asumir por el nombre de la funcionalidad ("motor de reglas") que debería comportarse igual para los dos tipos de solicitud — ES-01 y ES-02 tienen mecanismos distintos, aunque compartan la misma tabla requests.
- Cuando una afirmación de la IA sobre permisos/comportamiento del sistema no se verificó explícitamente contra el código en el momento en que se dijo, vale la pena volver a confirmarla antes de construir un plan de pruebas sobre ella.

Entrada — 20 de agosto de 2026 (continuación — verificación de la notificación por correo de ES-03 y traducciones faltantes en el correo)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Verificación en vivo de la notificación por cambio de estado (ES-03) y corrección de textos sin traducir en el correo generado

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la terminal para inspeccionar código, cola de trabajos y logs

1. Qué se le consultó a la IA

El equipo preguntó cómo funciona la notificación por correo de ES-03 ("notificación por correo en cada cambio de estado"), que en una sesión anterior (14 de agosto) había quedado registrada como explícitamente pospuesta a un tercer avance. Al confirmar que sí existía implementada en el código actual, se le pidió a la IA una guía paso a paso para verla funcionar en vivo, y luego que revisara el resultado real una vez el equipo la probó.

2. Qué encontró la IA antes de actuar

1. Antes de dar por hecho que la notificación ya no funcionaba según lo documentado, buscó en el código actual (grep por Mail::Notification::Mailable) y encontró que sí existe una implementación completa: RequestNotifierInterface (puerto en Domain), EloquentRequestNotifier (adaptador en Infrastructure) y RequestStatusChangedNotification (la notificación de Laravel en sí, con ShouldQueue). Esto contradice el estado registrado el 14 de agosto la IA lo señaló explícitamente como una actualización a esa nota anterior en vez de mezclarla o de asumir que su información previa seguía vigente.

2. Verificó el disparador exacto en ChangeRequestStatusUseCase.php: se manda solo si `$previousStatus !== $newStatus`, es decir, cuando el revisor solo actualiza la fecha estimada sin mover el estado, no se dispara coincide con la letra literal de "en cada cambio de estado", no "en cada guardado".

3. Revisó .env y confirmó MAIL_MAILER=log (el correo se escribe en storage/logs/laravel.log en vez de enviarse a un servidor real) y QUEUE_CONNECTION=database (necesita un worker corriendo para procesar el trabajo encolado) y confirmó que el propio composer run dev del proyecto ya levanta php artisan queue:listen, así que no había que configurar nada nuevo.

4. Tras la verificación en vivo (el equipo cambió el estado de una solicitud y corrió php artisan queue:work --once), la IA leyó el contenido real del correo generado en el log no solo el mensaje "DONE" de la cola y notó que, aunque los valores de estado sí salían en español ("Pendiente de revisión", "Denegada"), el

resto del texto del correo (saludo, cuerpo, firma) estaba en inglés. Diagnosticó la misma causa raíz que el filtro de la bandeja corregido antes en el mismo día: frases pasadas a ___() sin su entrada correspondiente en lang/es.json.

5. Antes de tocar el diccionario, corrió un script propio de una sola pasada sobre todo el proyecto (no solo el archivo de la notificación) para no repetir el mismo error de alcance incompleto de la corrección anterior. Encontró 117 frases sin traducir en total, pero clasificó correctamente que 108 pertenecen a pantallas del starter kit compartido (login, registro, 2FA, passkeys, configuración de cuenta) — fuera del alcance del módulo del equipo — y solo 9 pertenecen al código propio (RequestStatusChangedNotification.php). Se lo presentó al equipo con esa distinción antes de decidir qué traducir, en vez de traducir las 117 de una vez.

3. Qué se aceptó de la respuesta de la IA

- El diagnóstico de que la implementación de ES-03 sí existe y actualiza lo registrado el 14 de agosto (donde se documentó como pendiente) corrección explícita del propio diario, no una nota nueva aislada.
- La guía de verificación en vivo: cambiar estado → confirmar el job en la tabla jobs → php artisan queue:work --once → leer storage/logs/laravel.log.
- Las 9 traducciones agregadas a lang/es.json para el correo de notificación (saludo, cuerpo del mensaje, tipos de solicitud, estado anterior/nuevo, fecha estimada, cierre).
- La decisión de no tocar las 108 frases del starter kit compartido, siguiendo el mismo límite de alcance ya establecido en sesiones anteriores.

4. Qué se rechazó y por qué

- Se rechazó (implícitamente, al no pedirlo) traducir las 108 frases del starter kit coherente con el límite de alcance ya documentado: solo ES-01–ES-04 cuentan para la nota, no la plantilla compartida de autenticación/perfil.
- La IA hizo la corrección de las 9 frases sin esperar una segunda confirmación explícita del equipo (a diferencia de su costumbre habitual de solo explicar y dejar que el equipo edite) el equipo notó esto ("qué haces") y, tras la aclaración, confirmó que sí quería mantener el cambio. Queda registrado como una desviación puntual de la norma de "explicar, no escribir directamente" que debe volver a pedirse explícitamente la próxima vez, no asumirse por precedente de una sesión anterior donde sí se autorizó.

5. Qué hubo que corregir o verificar manualmente

- No se dio por bueno el mensaje "DONE" de queue:work como prueba suficiente de que la notificación funcionaba se leyó el contenido real del correo en el log para confirmar que los datos (nombre del estudiante, curso,

estados, fecha) eran correctos, y fue precisamente esa lectura la que reveló el problema de traducción que un simple "sin errores" no hubiera mostrado.

- Se validó la sintaxis de lang/es.json con json_decode después de la edición (mismo paso de verificación que ayer), dado que una de las frases nuevas contiene comillas dobles literales dentro del string ("My requests") que debían quedar correctamente escapadas en JSON.

6. Qué se aprendió del proceso

- La documentación de sesiones anteriores en este mismo diario puede quedar desactualizada si el trabajo avanza en sesiones intermedias no registradas aquí vale la pena, ante cualquier afirmación de "esto está pendiente/no existe", volver a verificarla contra el código actual en vez de repetirla de memoria, como se hizo aquí antes de responder la pregunta del equipo.
- Confirmar solo que un job de cola terminó sin error ("DONE") no es lo mismo que confirmar que su contenido es correcto — hay que leer el efecto real (en este caso, el texto del correo) para encontrar problemas que no lanzan una excepción, como una traducción faltante.
- El mismo tipo de bug (texto sin traducir) puede repetirse en partes distintas del código con la misma causa raíz (__() sin entrada en el diccionario) una vez detectado el patrón, vale la pena barrer todo el proyecto de una vez en lugar de ir corrigiendo instancia por instancia según van apareciendo, pero también distinguir con cuidado qué está en el alcance del equipo antes de tocarlo.
- Una autorización para editar código dada en una sesión no debe generalizarse automáticamente a ediciones posteriores dentro de la misma conversación, aunque el patrón del problema sea idéntico el equipo señaló este punto explícitamente, y queda como recordatorio para pedir confirmación en cada corrección de código, no asumirla por precedente inmediato.

Entrada — 20 de agosto de 2026 (continuación — prueba de envío real de la notificación ES-03 con SMTP de Gmail, y corrección de marca/firma en el correo)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Verificación de extremo a extremo de la notificación de ES-03 usando un envío real por SMTP (no solo el log), y corrección de la marca ("Laravel") y el cierre en inglés ("Regards,") que aparecían en el correo recibido

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la terminal, al proceso de desarrollo local (Herd) y al documento oficial de especificación del proyecto

1. Qué se le consultó a la IA

Hasta la entrada anterior, la verificación de ES-03 se había hecho leyendo el contenido del correo directamente del log (MAIL_MAILER=log). El equipo quiso ir un paso más allá y comprobar el envío real: configurar el proyecto para mandar el correo de verdad por SMTP a una cuenta de Gmail personal del equipo y verlo recibido como lo vería un estudiante real, de forma manual y guiada paso a paso.

2. Qué encontró e hizo la IA

1. Explicó qué variables de .env controlan el envío (MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM_ADDRESS) y que Gmail exige una "contraseña de aplicación" (App Password) en vez de la contraseña normal de la cuenta.

2. Tras el primer cambio de estado de prueba, el correo no llegó. Diagnosticando desde el log, la IA notó que el envío seguía usando el driver log y la dirección por defecto (hello@example.com), pese a que él .env ya tenía los valores nuevos. La causa: el proyecto levanta php artisan serve y php artisan queue:listen como procesos de larga duración vía composer run dev, y Laravel carga las variables de entorno en modo "immutable" — un proceso que ya arrancó con los valores viejos no relee él .env aunque se edite, y ese comportamiento se propaga incluso a los procesos hijo que queue:listen lanza por cada trabajo, porque heredan el entorno del proceso padre. Reiniciar el servicio de Herd no bastó, porque esos procesos no son parte de Herd.

3. Un primer intento de reiniciar solo los procesos PHP dejó vivos sus procesos padres de Node.js (npx concurrently, vite) de una ejecución anterior de composer run dev, que siguieron corriendo en segundo plano. Hubo que localizarlos explícitamente con Get-CimInstance Win32_Process y cerrarlos también antes de levantar un stack realmente limpio.

4. Con el proceso ya limpio, el envío sí intentó conectarse a Gmail, pero el log mostró un rechazo SMTP 535 ("Username and Password not accepted"): la contraseña en .env seguía siendo el valor de ejemplo ("xxxx xxxx xxxx xxxx") que la IA había puesto como plantilla en la guía, no la App Password real generada por el equipo. Se identificó el error leyendo el mensaje de la excepción en el log, no por ensayo y error.

5. Una vez confirmado el envío y recibido el correo, el equipo notó que el remitente y la firma decían "Laravel" en vez del nombre del sistema, y pidió corregirlo. La IA rastreó ambos textos a una sola causa: la variable APP_NAME del .env seguía en su valor por defecto de instalación (Laravel), de la cual dependen tanto MAIL_FROM_NAME (config/mail.php:115) como el nombre de marca que Laravel usa en la plantilla de correo. El cierre "Regards," resultó ser un texto fijo que la plantilla de Laravel imprime con @lang('Regards,') cuando la notificación no define un ->salutation() propio no algo ya implícito del código de RequestStatusChangedNotification.

6. Antes de asumir que "SIGA" era el nombre correcto, se revisó el documento oficial de especificación del proyecto (Proyecto_4_Solicitudes_Estudiantiles.docx) para confirmar si exigía un formato o marca específica para el correo de ES-03. El documento no lo exige ES-03 solo pide que se notifique por correo en cada cambio de estado, pero sí nombra el sistema en su encabezado como "Sistema Integrado de Gestión Académica y Docente", de donde sale el acrónimo SIGA.

3. Qué se aceptó de la respuesta de la IA

- Cambiar APP_NAME=Laravel a APP_NAME=SIGA en .env local únicamente, ya que .env está en .gitignore y no se sube al repositorio.
- Agregar "Regards,": "Saludos," a lang/es.json, siguiendo el mismo mecanismo de traducción por JSON ya usado para el resto del correo de ES-03 en la entrada anterior este sí es un cambio de código real.
- El diagnóstico completo de por qué el .env no se aplicaba (caché de entorno a nivel de proceso, no de configuración de Laravel) antes de intentar cualquier solución.

4. Qué se rechazó y por qué

- No se inventó ni asumió una plantilla de correo "oficial" al confirmar que el documento de especificación no la exige, se dejó claro al equipo que SIGA/Saludos es una decisión de pulido propia del equipo, no un requisito literal del SRS, para no presentarlo como algo que no es.
- La cuenta de Gmail personal usada para la prueba y su App Password se mantuvieron fuera de cualquier archivo del repositorio y de este mismo diario

se trataron como datos de prueba locales, no como algo que deba quedar documentado ni versionado.

5. Qué hubo que corregir o verificar manualmente

- Después de confirmar que el correo llegaba bien, se revirtió manualmente el estado de prueba a como estaba antes: MAIL_MAILER de vuelta a log (y el resto de los valores MAIL_* a sus defaults) y el correo del usuario estudiante de prueba de vuelta a su valor original en la base de datos — para no dejar la cuenta personal ni credenciales reales de Gmail como el estado persistente del entorno local del equipo.
- Se verificó explícitamente, antes de dar la corrección por buena, que .env no está trackeado por git (git check-ignore -v .env) y que el único archivo modificado sujeto a commit era lang/es.json para asegurar que ningún dato personal terminara expuesto en el historial del repositorio.

6. Qué se aprendió del proceso

- El modo "immutable" de Dotenv en Laravel tiene una consecuencia poco intuitiva en desarrollo local: procesos de larga duración (artisan serve, queue:listen) no releen el .env al editarlo, y ese comportamiento se hereda incluso en los procesos hijo que lanzan un php artisan config:clear no soluciona esto, hace falta matar y volver a levantar los procesos por completo.
- Al matar procesos de desarrollo para forzar una relectura de configuración, no basta con matar el proceso hijo visible (el php.exe de artisan serve); hay que revisar también sus procesos padres (los de node.exe/npm que los lanzaron), o quedan corriendo en segundo plano compitiendo por el mismo puerto.
- Antes de personalizar textos o marca en una salida generada (como el correo de ES-03), vale la pena confirmar contra el documento de especificación oficial si existe un requisito real detrás, en vez de asumir que cualquier mejora percibida por el equipo es parte del alcance formal del proyecto.

Entrada — 20 de agosto de 2026 (continuación — columnas Estudiante/Curso mostrando IDs crudos en la bandeja de Docencia, y datos de prueba para verificarlo)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Corrección de la tabla "Gestión de solicitudes" (ES-04) para mostrar nombre de estudiante y curso en vez de sus IDs numéricos, decisión de diseño sobre qué tan detallada debe verse cada fila, y creación de datos de prueba para observar el resultado con varios estudiantes distintos

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la terminal y a la base de datos local

1. Qué se le consultó a la IA

El equipo compartió una captura de la bandeja de Docencia (/solicitudes) donde la columna "Estudiante" mostraba 1 en cada fila en vez del nombre del estudiante, y preguntó qué archivo había que modificar para corregirlo.

2. Qué encontró la IA antes de actuar

1. Ubicó la vista de la tabla (resources/views/requests/request/livewire/request-component.blade.php) y confirmó que la fila imprimía directamente `$request->studentId()` (línea 90) y `$request->courseId()` (línea 100) los IDs numéricos crudos del modelo de dominio, sin resolver a nombre.

2. Revisó RequestComponent.php y encontró que la resolución de ID a nombre ya existía en el propio componente: `studentLabelsById()` ("Nombre Apellido (cédula)") y `courseLabelsById()` ("CÓDIGO — Nombre"), ya usados correctamente en el modal de detalle (`openViewModal()`) y en las exportaciones PDF/Excel (`exportableRows()`). El problema era que `render()` el método que alimenta la tabla en pantalla nunca pasaba esos dos arrays a la vista, así que el blade caía a imprimir el ID crudo sin tener otra opción.

3. Siguiendo la costumbre de este equipo (explicar antes de escribir código, salvo autorización explícita), presentó el diagnóstico y la ubicación exacta del cambio (archivo, línea, código a escribir) sin aplicarlo, hasta que el equipo pidió explícitamente "aplícalos para refrescar y mirar los cambios".

3. Qué se aceptó de la respuesta de la IA

- El diagnóstico de causa raíz (falta de paso de datos a la vista, no un bug en la lógica de resolución en sí).
- Aplicar el cambio: pasar `studentLabels/courseLabels` desde `render()` y usarlos en las dos líneas del blade con fallback al ID crudo

(`$studentLabels[$request->studentId()] ?? $request->studentId()`), una vez autorizado explícitamente.

- La recomendación de diseño cuando el equipo preguntó si convenía una columna "Identificación" separada: la IA señaló el trade-off (más claro vs. una séptima columna en una tabla ya de seis) y recomendó separar, pero el equipo decidió una tercera opción omitir la cédula del todo en esa fila y dejar solo el nombre, manteniéndola en el modal de detalle y las exportaciones.
- Para esa tercera opción, la IA no modificó `studentLabelsByld()` (compartido con modal/exportaciones), sino que agregó un método nuevo, `studentNamesByld()`, exclusivo para la columna de la tabla y verificó antes de aplicarlo que la búsqueda de la bandeja (`EloquentRequestRepository::baseQuery()`) ya filtra por nombre, apellido y cédula directamente contra la tabla `students`, independiente de qué texto se muestre en pantalla, así que omitir la cédula de la vista no rompía la búsqueda por cédula que el equipo quería conservar.

4. Qué se rechazó y por qué

- No se creó una columna "Identificación" separada el equipo, tras escuchar el trade-off, prefirió la fila más simple (solo nombre) en vez de una columna adicional.
- Cuando el equipo pidió crear "varios perfiles de estudiantes" para probar la tabla, la IA no creó perfiles nuevos: encontró que ya existían 10 expedientes de estudiantes sin ninguna solicitud asociada (creados manualmente por el equipo en alguna sesión anterior, no por ningún seeder del código) y los usó para crear solicitudes de prueba en vez de duplicar datos.
- Ante la frase ambigua del equipo "cada vez que inicie sesión como estudiante y cree una solicitud, esta me recargue con un nombre", la IA no asumió que se pedía una función donde el portal del estudiante rotara de identidad en cada solicitud señaló explícitamente que eso no sería correcto para un sistema real (una cuenta de estudiante representa siempre a la misma persona) y usó una pregunta de aclaración en vez de adivinar o de implementar algo potencialmente equivocado. El equipo confirmó que solo quería ampliar los datos de prueba a 10 estudiantes con solicitud, no esa función.

5. Qué hubo que corregir o verificar manualmente

- Se verificó (`git status`) que los 20 registros de solicitudes de prueba creados en dos rondas viven únicamente en la base de datos local no hay ningún seeder ni migración tocado así que no aparecen como cambios pendientes de commit, a diferencia del fix de código real (`request-component.blade.php` y `RequestComponent.php`), que sí quedó listo para commit por separado.

- Antes de insertar las solicitudes de prueba directamente vía Eloquent (sin pasar por el formulario del portal), se revisaron las migraciones de students y requests para confirmar qué columnas son realmente obligatorias a nivel de base de datos, evitando insertar datos que violaran una restricción NOT NULL o una clave foránea inexistente.

6. Qué se aprendió del proceso

- Antes de "crear datos nuevos" para una prueba, vale la pena revisar primero si ya existen datos utilizables sin usar en este caso, 10 expedientes de estudiantes reales sin solicitudes, en vez de generar duplicados que solo agregarían ruido a la base de datos.
- Una petición del equipo escrita de forma apurada o ambigua puede sonar a una función razonable a primera lectura, pero conviene parafrasearla como una pregunta de opción múltiple concreta antes de construir algo en este caso evitó implementar una noción de "identidad rotativa" para el estudiante que no tenía sentido en el dominio del sistema.

Entrada — 20 de agosto de 2026 (continuación — reemplazo del panel de filtros de ES-04 por un buscador único, decisión de Docencia)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Eliminación completa del panel de filtros (Tipo, Estado, Programa, fechas) de la bandeja de Docencia y extensión del buscador de texto para que cubra esas mismas dimensiones, por decisión estética confirmada con el docente

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal

1. Qué se le consultó a la IA

En la misma sesión, el equipo primero mostró una captura del panel de filtros de la bandeja y comentó que se veía "feo" o mejor dicho no muy agradable para los usuarios, proponiendo eliminarlo y que el mismo cuadro de "Buscar" funcionara como filtro. Tras la recomendación de la IA (ver más abajo, punto 2), el equipo lo comentó con el docente del curso, quien decidió explícitamente por estética que el buscador debía actuar como filtro único por estudiante, tipo, curso, estado y fecha de recepción, y pidió aplicar ese cambio.

2. Qué encontró la IA antes de actuar

1. En la primera consulta (antes de la decisión del docente), la IA no aceptó de inmediato quitar los filtros: señaló que el documento de especificación del proyecto (Proyecto_4_Solicitudes_Estudiantiles.docx, ES-04) exige explícitamente una bandeja "filtrable por tipo, carrera, estado y fecha de recepción", y que depender de un solo cuadro de texto libre para adivinar esas cuatro dimensiones era técnicamente frágil. Propuso una alternativa intermedia (panel colapsable detrás de un botón "Filtros") que resolvía la queja estética sin tocar la funcionalidad, y la aplicó tras autorización explícita del equipo.

2. Cuando el equipo volvió con la decisión ya tomada por el docente, la IA no insistió en el mismo argumento procedió a implementar el cambio pedido, pero dejando explícito qué se perdía en el camino.

3. Antes de tocar el buscador, revisó cómo se muestran actualmente el tipo y el estado en pantalla: son valores en inglés en la base de datos ('Requirement Waiver', 'Approved', etc.) traducidos a español solo en la vista vía __(). Para que buscar "Aprobada" funcionara, el WHERE de la consulta tenía que comparar contra la etiqueta traducida, no contra el valor crudo de la columna así que construyó el mapeo de etiquetas dentro de EloquentRequestRepository::baseQuery() usando las mismas claves de lang/es.json ya existentes (para no duplicar un diccionario nuevo).

4. Para la fecha, evitó una solución no portable (funciones SQL específicas de un motor) dado que el proyecto corre en MySQL localmente, pero `.env.example` documenta SQLite como default usó `\DateTime::createFromFormat('Y-m-d', ...)` para validar que el término de búsqueda sea una fecha exacta antes de aplicar `whereDate()`, evitando además intentar interpretar como fecha cualquier texto que no tenga ese formato.

3. Qué se aceptó de la respuesta de la IA

- Eliminar por completo el panel y el botón de filtros del blade, y todo el estado de Livewire que solo existía para alimentarlo (`filterType`, `filterStatus`, `filterCareerId`, `filterDateFrom`, `filterDateTo`, `showFilters`, los métodos `updatingFilterX()`, `clearFilters()`, `activeFilters()`, `careerOptions()`, y el `use CareerModel` que quedó sin ningún otro consumidor).
- Extender el buscador único para que haga match contra estudiante, curso, tipo (por etiqueta en español), estado (por etiqueta en español) y fecha exacta de recepción.
- Limpiar del diccionario (`lang/es.json`) las ocho entradas que solo usaba ese panel (All types, All statuses, Program, All programs, Received from, Received to, Filters, Clear filters) incluida "Filters", que la propia IA había agregado un par de turnos antes para el botón colapsable que ahora se descartó.

4. Qué se rechazó y por qué

- La IA no eliminó el parámetro `$filtres` de `RequestRepositoryInterface`, `ListRequestsUseCase` ni de `EloquentRequestRepository::baseQuery()` (las ramas que filtran por tipo/estado/carrera/fecha exacta siguen ahí, solo que ya ningún llamador les pasa nada). Lo señaló como una decisión deliberada de alcance quitar un parámetro del contrato de Dominio es una cirugía más invasiva que "quitar un botón de la pantalla", y lo dejó como pregunta abierta para el equipo en vez de decidir unilateralmente si ese código ahora inalcanzable debía borrarse también.
- No se agregó "programa/carrera" como dimensión de búsqueda, porque el docente solo mencionó estudiante, tipo, curso, estado y fecha la IA no lo dio por incluido, aunque el documento de especificación sí lo nombra como una de las cuatro dimensiones de filtro de ES-04, y lo dejó anotado como algo que el equipo tendría que pedir explícitamente si lo quiere cubierto también.

5. Qué hubo que corregir o verificar manualmente

- Tras cada eliminación de propiedad/método, se corrió `grep` sobre todo `src/` y `resources/` buscando cada nombre eliminado (`activeFilters`, `filterType`, `filterCareerId`, etc.) para confirmar que no quedaba ninguna referencia rota

encontró coincidencias en ValidationPrecedentComponent.php, pero verificó que era una propiedad del mismo nombre en una clase completamente distinta y no la tocó.

- Se validó la sintaxis de los tres archivos PHP/Blade tocados con php -l y la del lang/es.json con json_decode, antes de dar el cambio por terminado.
- Al revisar el resultado visual con el equipo, la propia IA notó (sin que se le pidiera) que la columna "RECIBIDA" de la tabla en realidad muestra estimatedResolutionDate() en vez de createdAt() un posible bug preexistente, no relacionado con este cambio. Lo señaló explícitamente en vez de corregirlo sobre la marcha, y quedó pendiente de que el equipo decida si se investiga.

6. Qué se aprendió del proceso

- Cuando una decisión de diseño la toma una autoridad externa al equipo (en este caso el docente, no solo preferencia del equipo), no tiene sentido que la IA repita el mismo argumento ya presentado antes corresponde implementar lo pedido y ser explícito sobre qué trade-off queda aceptado, no insistir en la recomendación original una vez que ya fue escuchada y superada por una decisión con más autoridad.
- Quitar una funcionalidad de la interfaz no es lo mismo que quitarla del sistema: fue necesario distinguir explícitamente qué capa tocar (Presentación: sí, borrar todo) de cuál no (Dominio/Aplicación: dejar intacto, con el parámetro ahora inalcanzable desde cualquier llamador real) y explicarle esa distinción al equipo en vez de decidirla en silencio.
- Un hallazgo incidental (el bug de la columna "RECIBIDA") encontrado mientras se verifica un cambio distinto vale la pena señalarlo de inmediato con el mismo nivel de detalle que cualquier otro hallazgo, en vez de guardárselo para después o corregirlo sin que el equipo lo pida.

Entrada — 21 de agosto de 2026 (sesión larga: fix de "Mis solicitudes", rediseño de convalidaciones a lote de cursos, catálogo académico real por carrera, nueva justificación de levantamiento, reducción de 4 a 3 estados, y limpieza de UI en Docencia)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Sesión de un solo día que cubrió un bug bloqueante, un rediseño de formulario acordado con Registro, la carga del catálogo académico real (dos carreras) con vinculación estudiante↔carrera, un campo nuevo de justificación institucional (SLR-002), la eliminación de un estado de solicitud redundante, y varios ajustes de consistencia en las pantallas de Docencia

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la terminal, la base de datos local (MySQL) y control del navegador (para verificar cada cambio con las cuentas de estudiante y Docencia antes de darlo por terminado)

1. Qué se le consultó a la IA

1. Al entrar como estudiante a "Mis solicitudes", la pantalla mostraba un error 500 ("Internal Server Error").
2. El equipo pidió rediseñar la UI de "Nueva solicitud de convalidación" con decisiones técnicas propias, para permitir varios cursos en un mismo envío, con un solo apartado de documentos que aceptara arrastrar y soltar.
3. Pidió cargar en la base de datos todos los cursos de nivel 1 a 6 de la oferta académica real (a partir de un PDF de Registro), "para tener variedad de datos disponibles" de cara a una demo.
4. Observó que el selector de curso de la solicitud mostraba cursos de todas las carreras mezclados, y preguntó si el docente/Docencia está vinculado a una carrera como el estudiante — de ahí surgió pedir que el sistema detectara automáticamente la carrera del estudiante autenticado y solo cargara sus propios cursos.
5. Pidió repetir el proceso de carga de catálogo para una segunda carrera (Ingeniería en Salud Ocupacional y Ambiente, a partir de un segundo PDF), con un segundo usuario de prueba para poder demostrar el filtro por carrera con dos perfiles distintos.
6. Pidió rediseñar la UI de "Nueva solicitud de levantamiento" (SLR-002 V.3) con decisiones técnicas propias, agregando el campo de justificación (5 categorías

institucionales fijas) y el aviso de "Notas importantes" con casilla de aceptación, conservando el resto del formulario tal cual.

7. Pidió reducir los estados de una solicitud de 4 a 3, eliminando "En revisión" por ser redundante con "Pendiente de revisión", según lo conversado con el personal de Registro.

8. Pidió quitar la columna "Resultado" de la tabla "Mis solicitudes" del estudiante, por ser repetitiva con "Estado".

9. Entrando como Docencia, señaló tres inconsistencias más para corregir: (a) el selector de carrera de "Nuevo precedente de convalidación" mostraba las 14 carreras del catálogo institucional en vez de solo las 2 con planes de estudio cargados; (b) el botón "Cerrar" de los modales de detalle de solo lectura es redundante con la × de la esquina (y en el caso de Convalidaciones, el botón "Editar" tampoco debería estar ahí); (c) "Reglas de Levantamiento" no tenía un botón de "Ver" como las demás pantallas, y su selector de curso también debía exigir una carrera primero; y por último pidió renombrar "Resultado" → "Estado" en Convalidaciones y "Activo" → "Estado" en Reglas de Levantamiento (dejando el valor del badge como "Activo"/"Inactivo"), con el badge de "Inactivo" en rojo en vez de gris.

2. Qué encontró la IA antes de actuar

1. El error 500 venía de `NagerDateHolidayCalendar::holidaysForYear()`: la llamada a `Cache::remember()` usaba el argumento nombrado `now:`, pero el parámetro real de `Illuminate\Cache\Repository::remember()` se llama `$ttl`, no `$now` — un Error de PHP por argumento nombrado inexistente, no una excepción de negocio.

2. Antes de tocar el formulario de convalidación, la IA hizo preguntas de alcance explícitas (con `AskUserQuestion`) en vez de asumir: si "varios cursos en la misma resolución" implicaba un agregado de dominio nuevo (una "Resolución" compartida) o simplemente varias Request independientes creadas en un solo envío. El equipo, tras consultar con Registro, confirmó la segunda opción sin migración de base de datos ni concepto de lote en el dominio.

3. Al cargar el catálogo académico, encontró que ya existían 5 cursos "placeholder" (códigos inventados, no reales) con 20 solicitudes, 2 reglas de dispensa y 1 precedente ya enganchados a ellos. En vez de borrarlos sin más, preguntó explícitamente cómo proceder; el equipo eligió borrar todo en cascada, así que la IA verificó primero cada restricción de llave foránea relevante (`requests.course_id` es `restrictOnDelete`, `validation_precedents.course_id`

también, `waiver_rules.course_id` es `cascadeOnDelete`) para borrar en el orden correcto sin que la base de datos rechazara la operación a medias.

4. Para el filtro de curso por carrera, revisó primero si existía algún vínculo estudiante↔carrera en el esquema antes de proponer una solución: encontró que la tabla `student_study_plan` ya existía desde el inicio del proyecto, pero estaba completamente vacía. La solución no fue "agregar una función nueva", sino conectar una relación que ya estaba en el diseño original.

5. Antes de escribir el campo de justificación, revisó que no existiera ya ninguna columna equivalente en `requests`, y qué lugares (modal de detalle del estudiante y de Docencia) necesitarían mostrar el dato nuevo para que no quedara capturado pero invisible para el revisor.

6. Antes de eliminar "En revisión", verificó en el código que ese estado no tuviera ninguna regla de negocio distinta de "Pendiente de revisión" (sin transición obligatoria, sin permiso especial, 0 filas usándolo en la base de datos) y que el propio README (ES-03) lo listaba como parte de la especificación original para poder señalarlo como una decisión que sustituye al documento base, no un descuido.

7. Antes de quitar el botón "Cerrar"/"Editar" de los modales de solo lectura, confirmó que el componente `x-ui.modal` es compartido por toda la aplicación, así que el cambio se hizo ahí (ocultar el pie del modal si no tiene contenido) en vez de en cada pantalla, beneficiando a futuros modales de solo lectura sin tocarlos uno por uno.

3. Qué se aceptó de la respuesta de la IA

- El fix de `NagerDateHolidayCalendar` (pasar el `$ttl` de forma posicional).
- El rediseño completo de "Nueva solicitud de convalidación": arreglo editable de hasta 8 líneas de curso (curso UTN + nombre externo + universidad), un solo dropzone de archivos con "arrastrar y soltar", y al enviar una `Request` independiente por cada línea de curso, cada una con su propia copia de los documentos adjuntos.
- La carga completa del catálogo de Ingeniería del Software (27 cursos, niveles 1-6) y de Ingeniería en Salud Ocupacional y Ambiente (32 cursos, niveles 1-6), cada uno con su propio plan de estudios, y la creación de `estudianteISW@gmail.com` / `estudianteISOA@gmail.com` como perfiles de demo, cada uno matriculado en su propia carrera vía `student_study_plan`.
- El filtro automático de `StudentRequestComponent::courseOptions()` por la(s) carrera(s) del estudiante autenticado, con reserva a mostrar el catálogo

completo si el estudiante no tiene matrícula registrada (para no dejar nunca un selector vacío).

- El campo `waiver_justification` (5 categorías fijas, migración nueva, mostrado en ambos modales de detalle) y la casilla "He leído y acepto las notas importantes anteriores" validada como obligatoria pero nunca persistida, por ser solo evidencia de que el estudiante vio el aviso.
- La reducción a 3 estados de solicitud, incluyendo la migración que remapea cualquier fila 'In Review' preexistente antes de estrechar el ENUM.
- Quitar la columna "Resultado" de la tabla del estudiante (dejando el dato disponible en el modal de detalle, sin perderlo).
- En Docencia: filtrar el selector de carrera de Convalidaciones a solo las 2 con cursos cargados; ocultar el pie de los modales de solo lectura; agregar el botón "Ver" + modal de detalle a Reglas de Levantamiento; exigir una carrera antes de cargar el selector de curso en Convalidaciones y en Reglas de Levantamiento (con aviso en rojo bajo el campo); renombrar "Resultado"→"Estado" y "Activo"→"Estado" reutilizando la misma traducción ya usada por las solicitudes; y el badge "Inactivo" en rojo (status-badge negative) igual que "Denegada".

4. Qué se rechazó y por qué

- No se creó un agregado de dominio nuevo ("Resolución compartida") para agrupar los cursos de una misma solicitud de convalidación se decidió explícitamente que cada curso genera su propia Request, revisable por separado en el inbox de Docencia, evitando una migración y un rediseño de dominio no pedidos por Registro.
- No se le pidió al estudiante elegir manualmente su carrera en el formulario de convalidación la IA recomendó en contra de esa idea (aunque el equipo la propuso primero) porque el sistema ya puede detectarla automáticamente vía `student_study_plan`, y pedírsela al estudiante habría sido menos realista y un paso manual innecesario.
- No se construyó ninguna pantalla de "matrícula de estudiantes" para poblar `student_study_plan` a futuro el equipo confirmó que el alcance del proyecto es solo el módulo de Solicitudes, y que esa matrícula se asume ya cargada por un sistema externo (Registro), igual que en el sistema real.
- No se tocó el formulario de Docencia para crear solicitudes de dispensa manualmente (RequestForm/RequestComponent) al agregar la justificación el pedido fue explícitamente sobre la pantalla del estudiante, y el campo quedó opcional (null por defecto) para no forzar un cambio no solicitado en el flujo de Docencia.

- No se eliminó la traducción "All careers" de golpe sin verificar — se confirmó primero que su único uso en toda la vista era exactamente el que se estaba reemplazando, antes de reutilizar esa misma entrada para el nuevo texto ("Seleccione una carrera").

5. Qué hubo que corregir o verificar manualmente

- Al implementar el envío de varios cursos con documentos compartidos, la primera prueba en el navegador falló con `UniqueConstraintViolationException` en la tabla `files`: `TemporaryUploadedFile::store()` de `Livewire` reutiliza el mismo nombre de archivo (derivado de un hash del propio temporal) en cada llamada, así que reutilizar el mismo archivo subido para la segunda `Request` chocaba con la restricción única (`disk, path`). Se corrigió generando un nombre aleatorio explícito (`Str::random(40)`) en cada copia, y se limpiaron manualmente los registros de prueba parciales que había dejado el primer intento fallido.
- Al ocultar el pie de los modales sin contenido, la primera versión (`$footer->isNotEmpty()` sin más) rompió con `ErrorException: Undefined variable $footer` en cualquier modal que nunca declarara ese slot blade no inicializa automáticamente un slot nombrado como objeto vacío si ningún llamador lo usa. Se corrigió con un `isset($footer)` antes de la comprobación.
- Cada cambio se verificó abriendo el navegador con las cuentas reales (`estudiantelSW@gmail.com`, `estudiantelSOA@gmail.com`, `docencia@gmail.com`), no solo revisando el código: se llenó y envió una solicitud de convalidación con 2 cursos reales, una de levantamiento con la justificación "c" seleccionada, y se alternó Activo/Inactivo en una regla de dispensa para confirmar visualmente el color del badge antes de restaurarla a su estado original.
- Antes de borrar los 5 cursos placeholder, se verificaron las 4 tablas con llave foránea hacia `courses` (`course_level`, `academic_records`, `validation_precedents`, `requests`) para no dejar un borrado a medias por una restricción no anticipada.
- Se corrieron `php -l`, `Pint` y `PHPStan` sobre cada archivo tocado en cada paso, y se corrió la suite completa de tests al final de la sesión — confirmando por comparación (`git stash temporal`) que las 2 fallas que persisten (`RequestTest::test_a_request_in_a_final_status_cannot_change_status_again` y `AuthenticationTest::test_login_screen_can_be_rendered`) ya existían antes de esta sesión y no están relacionadas con ninguno de estos cambios.
- Se verificó explícitamente, al cierre de la sesión, que ningún commit se había hecho durante todo el trabajo (`git status` limpio al inicio, 22 archivos modificados y 2 migraciones nuevas sin trackear al final) y se distinguió qué

de todo esto es código versionable (los 22 archivos + 2 migraciones) de qué es solo dato local no versionado (el catálogo de cursos, las carreras, los 2 usuarios de demo, sus matrículas), que no queda en ningún seeder y por tanto no sobrevive a un `migrate:fresh` en otra máquina.

6. Qué se aprendió del proceso

- `Livewire\Features\SupportFileUploads\TemporaryUploadedFile::store()` no genera un nombre aleatorio nuevo en cada llamada como sí lo hace `Illuminate\Http\UploadedFile::store()` base deriva el nombre del propio archivo temporal, así que reutilizar la misma subida para crear varios registros requiere `storeAs()` con un nombre explícito, no asumir que "guardar dos veces" produce dos copias independientes.
- Un slot nombrado de Blade (`<x-slot:footer>`) no existe como variable vacía por defecto dentro del componente si ningún llamador lo declara hay que comprobar `isset()` antes de preguntar si está vacío, a diferencia del slot por defecto (`$slot`), que sí siempre está definido.
- Antes de implementar "que el sistema detecte X automáticamente", vale la pena revisar si el esquema de base de datos ya tiene la relación necesaria sin usar (como pasó con `student_study_plan`) en vez de asumir que hace falta construir algo nuevo a veces la funcionalidad "nueva" es solo conectar un diseño que ya estaba ahí desde el principio.
- Cuando una decisión de UI se repite en varias pantallas parecidas (el patrón "seleccionar carrera primero, luego el curso"), conviene aplicarla de una sola vez a todas las pantallas afectadas apenas se identifica el patrón, en vez de esperar a que el equipo la pida pantalla por pantalla aunque en esta sesión sí se pidió explícitamente para cada una por separado.
- Verificar un cambio de UI solo leyendo el código no basta: dos de los bugs reales de esta sesión (la colisión de nombre de archivo y la variable `$footer` indefinida) solo aparecieron al probar el flujo completo en el navegador con datos reales, no durante la revisión estática ni con Pint/PHPStan.
- Datos de catálogo cargados directamente en la base de datos (carreras, cursos, matrículas, usuarios de demo) no dejan ningún rastro en git status vale la pena decirlo explícitamente al equipo al cierre de una sesión así, para que quede claro que ese trabajo no se pierde solo si en algún momento se decide formalizarlo en un seeder.

Entrada — 22 de agosto de 2026

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: Nueva acción "Ver detalle y documentos" en la bandeja de Docencia, con el expediente académico del estudiante sumado al modal de detalle

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso al código del proyecto (sin entorno local levantado en esta sesión)

1. Qué se le consultó a la IA

Se pidió agregar, en las acciones de la bandeja de solicitudes de Docencia, una nueva acción llamada "Ver detalle y documentos" que, además de mostrar el detalle de la solicitud y sus documentos adjuntos, también mostrara el expediente del estudiante.

2. Qué encontró la IA antes de actuar

1. La bandeja de Docencia (RequestComponent/request-component.blade.php) ya tenía una acción "Ver" con un modal de solo lectura (openViewModal) que mostraba el detalle de la solicitud y los documentos adjuntos, es decir, dos de los tres requisitos ya existían, solo faltaba el nombre solicitado y el expediente.

2. El botón "Ver" se renderiza con `<x-ui.row-actions>`, un componente compartido por Roles, Permisos, Precedentes de Convalidación, Reglas de Levantamiento y el propio portal del estudiante su título ("Ver") estaba fijo dentro del componente, así que cambiarlo directamente ahí habría renombrado el botón en las otras cinco pantallas sin que nadie lo pidiera.

3. El concepto de "expediente del estudiante" ya existía en el dominio, pero nunca se había mostrado en ninguna pantalla:

StudentAcademicProfileRepositoryInterface (con el comentario propio "expediente simulado") lee la tabla `academic_records` (curso, estado, nota, período académico) exclusivamente para que el WaiverEngine responda preguntas sí/no de forma automática ningún humano lo veía nunca directamente.

4. StudentModel ya tenía las relaciones `studyPlans()` (con el nivel actual vía `pivote`) y `academicRecords()` sin usar fuera del motor de reglas, así que no hacía falta construir infraestructura nueva para leer el expediente, solo cargarlas con `with()` y mapearlas a un arreglo para la vista, siguiendo el mismo patrón ya usado en `openViewModal()` para estudiantes/cursos/documentos.

3. Qué se aceptó de la respuesta de la IA

- Extender la acción y el modal "Ver" ya existentes en vez de crear una acción y un modal paralelos, ya que ambos requisitos (detalle + documentos) ya estaban resueltos ahí y compartían la misma autorización (RequestPolicy::view(), ya concedida a Docencia).
- Agregar un prop opcional viewLabel al componente compartido <x-ui.row-actions> (con __('View') como valor por defecto) para poder renombrar el botón únicamente en la bandeja de Docencia sin tocar Roles, Permisos, Precedentes, Reglas de Levantamiento ni el portal del estudiante.
- La nueva sección "Expediente del estudiante" dentro del modal existente: plan(es) de estudio con nivel actual, y el historial completo de academic_records (curso, estado con badge de color, nota, período), reutilizando StudentModel::with(['studyPlans', 'academicRecords.course', 'academicRecords.academicPeriod']) en vez de una consulta nueva por fuera del modelo ya existente.
- Las traducciones nuevas para los estados de academic_records que nunca se habían mostrado en una pantalla (Failed → "Reprobado", Credited by Equivalence → "Acreditado por equivalencia", Credited by Validation → "Acreditado por convalidación", Requirement Waived → "Requisito levantado").

4. Qué se rechazó y por qué

- No se reutilizó literalmente el estado Approved de una solicitud ("Aprobada", femenino, concuerda con "solicitud") como si fuera nuevo para el estado de un curso aprobado en el expediente (que en buen español concordaría como "Aprobado", masculino, con "curso"). Se aceptó conscientemente esa pequeña imprecisión de género en vez de introducir una clave de traducción duplicada con el mismo texto en inglés ("Approved") pero distinto valor en español, algo que Laravel no soporta con lang/es.json (una clave = un valor). Quedó señalado como un detalle cosmético menor, no como un error corregido a medias.
- No se tocó el permiso requests.view ni la policy: Docencia ya podía ver cualquier solicitud, así que no había nada que autorizar de nuevo para esta acción.
- No se creó una pantalla o ruta separada de "expediente del estudiante": el pedido fue mostrarlo junto al detalle y los documentos de la solicitud, no como una sección independiente del sistema.

5. Qué hubo que corregir o verificar manualmente

- El entorno de esta sesión no tenía vendor/ ni .env configurados, así que la IA no pudo levantar la aplicación ni verificar el cambio abriendo el navegador

con la cuenta de Docencia (docencia@gmail.com) a diferencia de sesiones anteriores donde sí se verificó cada cambio en vivo..

- Ante la falta de un entorno ejecutable, se corrieron las validaciones estáticas disponibles: `php -l` sobre `RequestComponent.php`, y `json_decode()` sobre `lang/es.json` para confirmar que las claves nuevas no rompieran el archivo (no había ninguna clave duplicada con las ~230 ya existentes).
- Se revisó a mano el balance de directivas Blade (`@if/@else/@endif/@foreach/@endforeach`) en el bloque nuevo del modal, ya que no hay un linter de Blade disponible en este entorno para detectarlo automáticamente.

6. Qué se aprendió del proceso

- Antes de agregar una acción o pantalla nueva, vale la pena revisar si ya existe algo parecido a medio camino (aquí, el modal "Ver" ya cubría dos de los tres requisitos pedidos) extenderlo evita duplicar autorización, consultas y markup que ya estaban resueltos y probados.
- Un componente de UI compartido por varias pantallas (`<x-ui.row-actions>`) no debe modificarse con un valor fijo nuevo solo porque una pantalla lo necesita conviene volverlo configurable con un valor por defecto que preserve el comportamiento de todos los demás llamadores.
- Un concepto de dominio que ya existe, pero solo se usa internamente (el "expediente simulado" que el motor de reglas consulta) puede tener ya toda la infraestructura de datos necesaria (modelos, relaciones) para exponerse en una pantalla antes de diseñar algo nuevo vale la pena revisar si el dato que se pide ya se está leyendo en algún otro punto del sistema, aunque sea con otro propósito.
- Sin un entorno ejecutable, la verificación de un cambio de UI queda necesariamente incompleta (sintaxis sí, comportamiento real no) es preferible decirlo con claridad en vez de dar a entender que el cambio fue probado igual que en sesiones con navegador disponible.

Entrada — 22 de agosto de 2026

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: Encabezado del modal de detalle reorganizado + nueva sección "Cursos a convalidar" con Reconocer/No reconocer por curso

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, sin entorno local levantado en esta sesión

1. Qué se le consultó a la IA

A partir de una referencia visual (mockup) compartida por el equipo, se pidieron dos cambios sobre el modal "Detalle de la solicitud" de Docencia:

1. Mover el badge de estado (hoy al final del modal, sección "Estado") a la cabecera, junto al título, y reordenar el cuerpo para que curso UTN + curso externo + universidad de procedencia aparezcan primero.
2. Agregar una sección "Cursos a convalidar" (solo para tipo Convalidación) con columnas Curso UTN/curso externo/universidad/código/créditos/resolución, un botón "Guardar datos externos", un campo de razón obligatorio solo para "No reconocer", y botones "Reconocer"/"No reconocer" por curso.

El equipo señaló explícitamente, en su propio mensaje, que el punto 2 implicaba una decisión de modelo de dominio (¿sigue siendo 1 solicitud = 1 curso, o hace falta una entidad nueva tipo RequestCourseLine para agrupar varios cursos por solicitud?) que debía definirse antes de tocar código, no asumirse.

2. Qué encontró la IA antes de actuar

1. `<x-ui.modal>` es compartido por 6 pantallas distintas (Roles, Permisos, Precedentes, Reglas de Levantamiento, Crear solicitud, este modal), sin ningún slot para contenido extra en la cabecera cualquier cambio ahí debía ser opt-in.
2. El dominio Request sigue siendo 1 solicitud = 1 curso externo, sin concepto de "envío"/lote que agrupe varias solicitudes creadas juntas.
3. Revisando el propio diario, la IA encontró que el 21 de agosto el equipo ya había decidido explícitamente no crear un agregado de dominio compartido ("Resolución") para agrupar los cursos de una misma convalidación cada curso genera su propia Request independiente. Dos de las tres opciones que el propio equipo planteaba en su mensaje (agrupar Requests hermanas, o crear RequestCourseLine) reabrían esa decisión ya cerrada. En vez de elegir una por su cuenta o ignorar el precedente, la IA presentó las tres opciones al equipo citando explícitamente esa decisión previa como contexto.

3. Qué se aceptó de la respuesta de la IA

El equipo escogió, ante las preguntas planteadas: (a) "1 fila = la propia Request" sin cambio de dominio y (b) "el resultado por curso es el mismo status que ya existe en Request". Con esas dos decisiones, se aceptó:

- Un slot opcional titleExtra en <x-ui.modal>, sin modificar el comportamiento de los otros 5 usos del componente.
- El reordenamiento del cuerpo (solo para Convalidación): curso UTN, curso externo, universidad, y luego estudiante/tipo.
- Dos columnas nuevas y nullable en requests (external_course_code, external_course_credits) vía migración puramente aditiva, y un SaveExternalCourseDataUseCase separado de ChangeRequestStatusUseCase para que "guardar datos externos" nunca escriba una fila de cambio de estado ni dispare notificaciones.
- Reutilizar changeStatus() (con un parámetro \$status opcional) para que los botones "Reconocer"/"No reconocer" disparen exactamente el mismo flujo que ya usaba el modal de revisión clásico: mismo comentario obligatorio al denegar, mismo historial, mismo correo de notificación.

4. Qué se rechazó y por qué

- No se creó ninguna entidad de dominio nueva ni un campo de resultado separado del status existente ambas alternativas fueron descartadas por el propio equipo al responder las preguntas.
- No se reutilizó la traducción existente Approved → "Aprobada" (femenino, concuerda con "solicitud") para el badge de "Resolución" de un curso se prefirió agregar etiquetas dedicadas "Reconocido"/"No reconocido" en vez de forzar una concordancia de género incorrecta ("Aprobada" no concuerda con "curso").

5. Qué hubo que corregir o verificar manualmente

- Sin vendor/ ni .env en este entorno, la IA no pudo levantar la aplicación ni probar el cambio en el navegador. Se limitó a validaciones estáticas: php -l sobre los 6 archivos PHP tocados, json_decode sobre lang/es.json, y conteo manual de directivas Blade (@if/@endif/@foreach/@endforeach) para confirmar que quedaran balanceadas, a falta de un linter de Blade disponible.
- Quedó pendiente que el equipo corra php artisan migrate y pruebe el flujo completo (guardar datos externos, reconocer, no reconocer sin razón para confirmar que bloquea) con la cuenta de Docencia antes de dar el cambio por cerrado.

6. Qué se aprendió del proceso

- Cuando una instrucción nueva del equipo choca con una decisión arquitectónica ya tomada y documentada en una sesión anterior, vale la pena citarla explícitamente al presentar las opciones así el equipo decide con memoria completa del proyecto, en vez de reabrir sin saberlo una discusión que ya se había cerrado.
- Reutilizar el mismo método de cambio de estado para dos puntos de entrada de UI distintos el modal de revisión clásico y los botones en línea de la nueva tabla de cursos evita mantener dos caminos paralelos hacia el mismo efecto de dominio, con el riesgo de que diverjan con el tiempo.

Entrada — 22 de agosto de 2026

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: Expediente del estudiante ampliado, dos estados de revisión intermedios de vuelta al enum, y subida de documentos desde Docencia

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, sin entorno local levantado en esta sesión

1. Qué se le consultó a la IA

A partir de una segunda referencia visual, el equipo pidió en un mismo mensaje:

1. Ampliar la sección "Expediente del estudiante" (hoy una tabla simple de Curso/Período/Estado/Nota) con datos de identificación (nombre completo, cédula/carné, correo institucional, badge de matrícula activa/inactiva), datos de carrera (carrera actual, cuatrimestre, plan de estudios), un resumen estadístico (Aprobadas/Reprobadas/Acreditadas/Promedio/Créditos) y una tabla ampliada (agregando "Cuatrimestre del Plan" y "Cursado en"), todo dentro de un encabezado colapsable con contador y chevron.

2. Rediseñar el modal de revisión: la fecha estimada pasa a tener su propio botón "Guardar fecha", el estado deja de ser un <select> y pasa a ser botones (incluyendo un estado nuevo, "Verificada por Registro", y el regreso de "En revisión" eliminado la sesión anterior por redundante), los documentos adjuntos cambian a un formato de dos líneas, y se agrega la posibilidad de subir un documento nuevo desde Docencia.

El equipo marcó explícitamente que el estado nuevo y la subida de documentos eran cambios de dominio/infraestructura ("no solo de vista"), no una simple maqueta.

2. Qué encontró la IA antes de actuar

1. Para el expediente: StudentModel/User ya tenían cédula, correo y matrícula disponibles; LevelModel::courses() (mismo pivote course_level que ya usa CourseModel::levels()) permitía resolver "Cuatrimestre del Plan" sin ninguna consulta nueva a la base de datos, tal como el propio equipo había sugerido revisar.

2. Para el flujo de estados: la IA detectó una tensión directa con una decisión ya tomada "En revisión" se había eliminado explícitamente el 21 de agosto por ser redundante con "Pendiente de revisión", y ahora se pedía de vuelta junto con un estado adicional. Además, el mockup de referencia no mostraba un botón "Aprobada", lo que el propio equipo interpretó como una posible señal de que

ese estado debía calcularse automáticamente cuando todos los cursos de la tabla del cambio anterior quedaran "Reconocidos" algo que dependía directamente de cómo había quedado resuelto el punto 2 de la entrada anterior (1 Request = 1 curso).

3. En vez de asumir una de las alternativas o implementar la subida de documentos sin confirmar alcance, la IA presentó ambas decisiones como preguntas explícitas al equipo antes de tocar el dominio.

3. Qué se aceptó de la respuesta de la IA

El equipo decidió (a) agregar los dos estados nuevos sin cálculo automático "Aprobada" se mantiene manual y (b) sí implementar la subida de documentos. Con eso, se aceptó:

- La extensión de `studentRecord()` con identificación, carrera/plan y un resumen agregado (conteos por status + promedio ponderado por créditos + créditos ganados/totales del plan, calculado en PHP a partir de datos ya cargados, sin tocar la base de datos), colapsada con Alpine (x-data/x-show) reutilizando la clase `.chevron-toggle` que ya existía para el menú lateral.
- Una migración puramente aditiva para los estados In Review y Verified by Registro (sin remapeo de datos, a diferencia de la migración que los había reducido la sesión anterior).
- Convertir el `<select>` de estado en 5 botones los 4 del mockup más "Aprobada" para no dejar sin forma manual de aprobar a las solicitudes de tipo Dispensa de requisito, que no pasan por la tabla de Reconocer/No reconocer del cambio anterior.
- Separar "Guardar fecha" en su propia acción (`AssignEstimatedResolutionDateUseCase`), para no ensuciar el historial de la solicitud con una fila de "cambio de estado" falsa cuando solo se corrige la fecha.
- La subida de documentos desde Docencia, reutilizando el mismo trait (`StoresRequestAttachments`) y el mismo repositorio (`RequestAttachmentRepositoryInterface`) que ya usan los formularios del estudiante, en vez de construir un mecanismo de almacenamiento paralelo.

4. Qué se rechazó y por qué

- No se creó el permiso nuevo (`requests.upload_document`) que el propio equipo sugirió como posible requisito para la subida de documentos se reutilizó la habilidad review ya existente, razonando que cualquier rol autorizado a revisar una solicitud es exactamente el rol que debería poder adjuntarle documentos. Crear un permiso nuevo habría requerido cambios de

seeder que la IA no podía verificar en este entorno sin base de datos disponible.

- No se quitó el botón "Aprobada" del selector de estado pese a que el mockup de referencia no lo mostraba se dejó explícitamente para no romper el único camino manual de aprobación de Dispensa de requisito.

5. Qué hubo que corregir o verificar manualmente

- Mismas limitaciones que la entrada anterior: sin vendor//.env, solo se pudo validar con `php -l` sobre los archivos PHP tocados y `json_decode` sobre las traducciones nuevas agregadas a `lang/es.json`.
- Se recontó a mano el balance de directivas Blade después de este cambio (`@if/@endif`: 17/17, `@foreach/@endforeach`: 7/7, `@error/@enderror`: 14/14) antes de darlo por completo, a falta de un linter de Blade en este entorno.

6. Qué se aprendió del proceso

- Cuando una decisión nueva reabre una decisión ya cerrada en una sesión anterior (aquí, resucitar "En revisión"), conviene señalarlo explícitamente como una tensión real con el historial del proyecto, en vez de implementarlo en silencio como si fuera la primera vez que se discute el equipo necesita saber que está revirtiendo algo, no solo agregando algo nuevo.
- Un mockup de referencia no siempre cubre todos los flujos existentes: en este caso, el mockup solo mostraba el camino de Convalidación (con su tabla de cursos y Reconocer/No reconocer), sin considerar que Dispensa de requisito no tiene ese mecanismo alternativo para llegar a "Aprobada". Antes de calcar un diseño de referencia al pie de la letra, vale la pena verificar que ningún camino funcional existente quede sin reemplazo.

Entrada — 22 de agosto de 2026

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: Separar la descarga de documentos adjuntos en vista previa y descarga forzada, a partir de un bug real encontrado por el equipo

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, sin entorno local levantado en esta sesión

1. Qué se le consultó a la IA

El equipo, revisando

`app/Http/Controllers/RequestAttachmentDownloadController.php`, encontró que su llamada a `RequestEntity::reconstitute()` no incluía el argumento `waiverJustification` y pidió agregar esa línea exacta. Poco después, en un mensaje separado con instrucciones de código detalladas, pidió dividir esa misma descarga en dos acciones `preview()` (el navegador muestra el archivo directamente) y `download()` (fuerza guardar, el comportamiento original) compartiendo la misma lógica de resolución y autorización, y reemplazar el bloque "Attached documents" del modal de detalle por dos botones (Vista previa/Descargar) en vez de un solo enlace.

2. Qué encontró la IA antes de actuar

Confirmó que `waiverJustification` es un parámetro obligatorio (sin valor por defecto) en la firma de `Request::reconstitute()`, agregado en una sesión anterior sin actualizar este controlador por lo que cualquier descarga de un documento adjunto, desde el lado de Docencia o del estudiante, fallaba con un error fatal de PHP por argumento faltante. No era un defecto cosmético: la ruta estaba completamente rota.

3. Qué se aceptó de la respuesta de la IA

La línea faltante, tal como la dio el equipo. Para la división `preview/download`: extraer la lógica compartida (resolver el archivo, confirmar que pertenece a un `Request`, autorizar contra `RequestPolicy::view()`) a un método privado `resolveFile()`, evitando duplicarla entre los dos métodos públicos nuevos. La ruta original mantuvo su nombre (`requests.request.attachment-download`), ahora apuntando a `download`; se agregó una ruta nueva `.../preview` → `preview`.

4. Qué se rechazó y por qué

No se usó una URL firmada para la vista previa mismo razonamiento ya documentado en el controlador original: el archivo solo se enlaza desde una pantalla ya autorizada, así que no hay un caso real donde alguien no autenticado necesite ese link.

5. Qué hubo que corregir o verificar manualmente

php -I sobre el controlador y el archivo de rutas, y conteo manual de balance de directivas Blade en el bloque de adjuntos reemplazado sin entorno ejecutable para probar la descarga/vista previa real en el navegador.

6. Qué se aprendió del proceso

Un parámetro nuevo agregado a un método de dominio con argumentos nombrados (reconstitute()) no rompe nada en tiempo de escritura si el editor no valida tipos solo se manifiesta en tiempo de ejecución, y puede quedar invisible durante días si nadie ejercita exactamente esa ruta. Tras agregar un parámetro requerido a un método de reconstitución de dominio, vale la pena buscar (grep) todos los sitios que lo llaman, no solo los que se tocaron en la misma sesión que lo agregó.

Entrada — 22 de agosto de 2026 (continuación 4)

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: Adjuntos visibles en el modal de revisión y rediseño de los botones de estado

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, sin entorno local levantado en esta sesión

1. Qué se le consultó a la IA

El equipo entregó tres instrucciones sobre el modal de revisión ("Cambiar estado"): (1) restylear el selector de estado el enum, la migración y los 5 botones ya estaban resueltos de una entrada anterior, solo faltaba cambiar los pills de status-badge por un toggle btn-primary/btn-secondary; (2) mostrar en este modal los documentos ya adjuntos a la solicitud (hoy solo visibles en el modal de detalle), reutilizando el mismo bloque de vista previa/descarga recién creado; y (3) subir el límite de tamaño de la subida de documentos de Docencia a 10MB.

2. Qué encontró la IA antes de actuar

Verificó que la primera instrucción ya estaba resuelta salvo el estilo visual del botón. Para los adjuntos, confirmó que `openReviewModal()` nunca cargaba la lista de documentos (solo el formulario de subida), y que la consulta para traerla ya existía duplicada dentro de `openViewModal()`. Para el límite de tamaño, revisó los dos patrones de validación de archivos ya existentes en el proyecto (`RequestForm`: `mimes:pdf,jpg,jpeg,png` en 5MB y 10MB según el formulario) y confirmó que ninguno acepta Word/Excel, pese a que el mensaje del equipo daba a entender que sí.

3. Qué se aceptó de la respuesta de la IA

Restylear los botones de estado a `btn-primary/btn-secondary`. Extraer la consulta de documentos a un método compartido `documentsFor()`, usado ahora por `openViewModal()`, `openReviewModal()` y `uploadReviewDocument()` (para refrescar la lista tras subir uno nuevo), evitando que dos copias de la misma consulta diverjan con el tiempo. Subir el límite a 10MB, alineado con el límite más generoso que ya existía en el formulario de convalidación del estudiante, en vez de mantenerlo en 5MB o inventar un tercer límite.

4. Qué se rechazó y por qué

No se agregaron tipos MIME adicionales (Word/Excel) como sugería el mensaje del equipo al no existir ese patrón en ningún formulario real del proyecto, agregarlo aquí habría introducido una tercera regla de validación inconsistente sin que el equipo lo confirmara explícitamente con ese detalle.

5. Qué hubo que corregir o verificar manualmente

php -l sobre RequestComponent.php, json_decode sobre las traducciones nuevas, y conteo de balance de directivas Blade (@if/@endif: 18/18, @foreach/@endforeach: 8/8) sin poder probar el flujo real en el navegador.

6. Qué se aprendió del proceso

Cuando el mismo dato (aquí, la lista de documentos de una solicitud) se necesita en más de un lugar de la interfaz, conviene extraerlo a un método compartido apenas aparece la segunda necesidad, en vez de esperar una tercera copia que lo justifique la primera duplicación ya es la señal. Antes de ampliar una validación siguiendo una instrucción que asume que cierto comportamiento "ya existe" en el código, vale la pena confirmarlo contra el código real en vez de asumir que la premisa es correcta.

Entrada — 22 de agosto de 2026

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: Historial de estados y stepper de avance en el modal de detalle

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, sin entorno local levantado en esta sesión

1. Qué se le consultó a la IA

El equipo señaló que la tabla `request_status_history` y su modelo ya existían, y que cada cambio de estado ya se registraba automáticamente en `ChangeRequestStatusUseCase::handle()` (estado anterior, estado nuevo, comentario, usuario, fecha) pero nunca se mostraba en ninguna pantalla. Pidió agregarlo al modal de detalle con un formato específico por entrada, marcando explícitamente como pendiente de decisión del equipo si además había que crear una fila inicial de historial en `CreateRequestUseCase` (algo que hoy no existe). Por separado, pidió un stepper visual de 3 pasos (Recibida / En trámite / Aprobada-o-Denegada) derivado del status, señalando también como decisión abierta si el tercer paso debía tener una etiqueta fija ("Resuelta") o dinámica según el resultado.

2. Qué encontró la IA antes de actuar

Confirmó el modelo `RequestStatusHistoryModel` y su relación `user()`. Para el stepper, verificó que las variables CSS que el equipo asumía que existían (`--positive`, `--pending`) en realidad no existen como tales en `app.css` — el sistema de diseño solo define clases (`.status-badge.positive`, etc.) respaldadas por otros nombres de variable (`--badgeCustomText` para verde, `--actionDeleteText` para rojo, `--actionEditText` para ámbar), así que hubo que mapear a los tokens reales en vez de usar los nombres literales que el equipo propuso.

3. Qué se aceptó de la respuesta de la IA

La consulta de historial (`statusHistoryFor()`), mostrada con el formato pedido (estado anterior → estado nuevo, badge de quién lo cambió o "Sistema", fecha, comentario aparte). El stepper de 3 pasos, con "En trámite" cumplido si el estado actual no es el inicial o si ya existe algún registro de historial (las dos condiciones que el equipo ofreció como alternativas, combinadas con "o"). Y, tras preguntarlo explícitamente al equipo con una pregunta directa, la etiqueta dinámica ("Aprobada"/"Denegada" con color) para el tercer paso, en vez de una etiqueta fija neutra.

4. Qué se rechazó y por qué

No se tocó `CreateRequestUseCase` para generar la fila inicial de historial que el mockup de referencia mostraba el equipo lo marcó explícitamente como una decisión pendiente, no como parte de esta instrucción. Hoy las solicitudes no muestran una entrada "(nueva) → Pendiente de revisión" en su historial, solo los cambios que Docencia ya hizo después de la creación.

5. Qué hubo que corregir o verificar manualmente

Mismas limitaciones de siempre en este entorno (sin `vendor/.env`): `php -l`, `json_decode` sobre las traducciones nuevas, y conteo de balance de directivas Blade, que llegó a 20/20 (`@if/@endif`), 9/9 (`@foreach/@endforeach`) y 2/2 (`@php/@endphp`) tras este cambio.

6. Qué se aprendió del proceso

Cuando una instrucción asume la existencia de algo específico del sistema de diseño (aquí, variables CSS con nombres concretos), vale la pena verificarlo contra el CSS real antes de usarlo literalmente el nombre de una clase (`.positive`) no siempre corresponde a una variable CSS del mismo nombre. Preguntar explícitamente por una decisión de diseño que el propio equipo marcó como abierta (el texto del tercer paso del stepper), en vez de elegir una opción "razonable" por su cuenta, evita tener que rehacer el trabajo si el equipo prefería la otra opción.

Entrada — 22 de agosto de 2026

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: Botones de "Cambiar estado a" del modal de revisión 4 sin "Aprobada" para Convalidación, 5 para Dispensa de requisito

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, sin entorno local levantado en esta sesión

1. Qué se le consultó a la IA

El equipo mostró un recorte adicional de la referencia (con datos de ejemplo, incluyendo un archivo con nombre generado por IA) señalando "falta esta sección", sin precisar qué exactamente. Ante la pregunta de la IA, aclaró que se trataba del orden y la cantidad de los botones de "Cambiar estado a": la referencia muestra 4 (Pendiente de revisión, Verificada por Registro, En revisión, Denegada), sin "Aprobada" y en un orden distinto al que ya existía.

2. Qué encontró la IA antes de actuar

Antes de aplicar el cambio, señaló la consecuencia que tendría quitar "Aprobada" sin condición: las solicitudes de tipo Dispensa de requisito no tienen ninguna otra vía en la interfaz para llegar a ese estado (a diferencia de Convalidación, que ya puede hacerlo vía "Reconocer" en la tabla "Cursos a convalidar") aplicar la referencia literalmente en ambos tipos habría dejado a Dispensa de requisito sin forma de aprobarse desde el modal.

3. Qué se aceptó de la respuesta de la IA

Mostrar los 4 botones de la referencia (sin "Aprobada", en su orden exacto) solo para solicitudes de tipo Convalidación, y mantener los 5 para Dispensa de requisito la opción que la IA recomendó al plantear la pregunta. Esto requirió agregar `$reviewingType` (poblado en `openReviewModal()` desde `$request->type()`) para que la vista supiera qué lista de botones renderizar.

4. Qué se rechazó y por qué

No se quitó "Aprobada" de forma universal como mostraba la referencia al pie de la letra se habría roto la única vía manual de aprobación para Dispensa de requisito, y el equipo, ante la pregunta explícita, prefirió la versión condicional en vez de aceptar esa regresión funcional.

5. Qué hubo que corregir o verificar manualmente

php -l y conteo de balance de directivas Blade (@if/@endif: 20/20, @foreach/@endforeach: 9/9) sin poder confirmar visualmente en el navegador que cada tipo de solicitud muestra el conjunto de botones correcto.

6. Qué se aprendió del proceso

Una instrucción de "hacer que coincida con la referencia" no siempre puede aplicarse literalmente si la referencia solo muestra un caso (aquí, una solicitud de Convalidación) y el sistema real tiene más de un flujo (Convalidación y Dispensa de requisito) compartiendo la misma pantalla. Antes de aplicar un cambio visual que quita una opción, vale la pena verificar si esa opción es la única vía funcional para algún caso de uso, en vez de asumir que "menos botones" es simplemente una mejora visual sin costo.

Entrada — 22 de agosto de 2026

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: Fusionar el modal "Review request" dentro de "Request detail" un solo modal en vez de dos

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal

1. Qué se le consultó a la IA

Tras varias rondas de ajustes al modal de revisión, el equipo ya probando la aplicación en su propio entorno mostró una captura del modal "Request detail" preguntando por qué no veía ciertos botones. Ante la pregunta de la IA para precisar cuáles, el equipo definió directamente lo que quería en vez de seguir aclarando el malentendido puntual: "ya sé qué quiero hacer, fusiona ambos y que funcionen como uno mismo, entonces agrega lo que hay en edit al view" unificar el modal "Review request" (botones de estado, fecha estimada, comentario, subida de documentos) dentro de "Request detail", eliminando el segundo modal.

2. Qué encontró la IA antes de actuar

Al planear la fusión, notó que el diseño anterior tenía un flujo de dos pasos para cambiar de estado: el botón de cada estado solo fijaba la propiedad `reviewStatus`; un botón "Confirmar" en el pie del modal de revisión, separado, era el que realmente llamaba a `changeStatus()`. El modal fusionado no conservaría ese pie de "Confirmar" propio si se copiaba el markup del selector de estado tal cual, los botones habrían quedado sin ningún efecto real (fijarían la propiedad pero nunca la aplicarían).

3. Qué se aceptó de la respuesta de la IA

Que cada botón de estado llame directamente a `changeStatus('X')` al hacer clic, aplicando el cambio de inmediato el mismo patrón de un solo clic que ya usan Reconocer/No reconocer en la misma pantalla, en vez de mantener un paso de confirmación separado que ya no tenía dónde vivir. Se eliminaron `showReviewModal`, `openReviewModal()`, `closeReviewModal()`, `reviewPrecedentResolution` y `reviewingDocuments` (todo duplicado o redundante tras la fusión), y `openViewModal()` pasó a ser el único lugar que inicializa el estado de revisión (estado, fecha, comentario, tipo/archivo de documento). Se quitó también el ícono de lápiz ("Editar") de la bandeja, dejando solo el de "Ver detalle y documentos" como única acción de entrada.

4. Qué se rechazó y por qué

No se intentó preservar un paso de "Confirmar" independiente (por ejemplo, agregando un botón fijo al final del modal fusionado) se prefirió consistencia total con el patrón de un solo clic que Reconocer/No reconocer ya habían establecido en la misma pantalla, evitando que convivieran dos formas distintas de aplicar un cambio de estado en un mismo modal.

5. Qué hubo que corregir o verificar manualmente

php -l, y conteo de balance de directivas Blade tras la fusión (@if/@endif: 22/22, @foreach/@endforeach: 8/8, @error/@enderror: 14/14, @php/@endphp: 2/2) — sin poder confirmar visualmente en el navegador que el flujo completo (clic en un botón de estado, guardar fecha, subir documento) funciona correctamente de punta a punta en este entorno.

6. Qué se aprendió del proceso

Al fusionar dos pantallas que antes tenían flujos de interacción distintos (una de dos pasos con confirmación aparte, otra de un clic), hay que revisar explícitamente si alguna de las dos piezas dependía de una estructura que la fusión elimina (aquí, el botón "Confirmar" del pie del modal que desaparece) copiar el markup sin ese análisis habría dejado un control visualmente presente pero funcionalmente muerto. Cuando el equipo interrumpe una pregunta de aclaración con una decisión ya tomada ("ya sé qué quiero hacer"), conviene ejecutar esa decisión directamente en vez de insistir en resolver primero la ambigüedad original, que quedó superada por la nueva instrucción.

Entrada — 22 de agosto de 2026

Contribuidor (commit de git): *maykel0 (maykel54547564844@hotmail.com)*

Tema: El enum de status nunca se actualizó de verdad en SQLite — las dos migraciones anteriores solo evitaban el error de sintaxis, no arreglaban la restricción real

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con el equipo probando en su propio entorno local (SQLite)

1. Qué se le consultó a la IA

Al hacer clic en "Verificada por Registro" en el modal ya fusionado, el equipo reportó un `Illuminate\Database\QueryException`. La primera captura mostraba `no such column: external_course_code` resuelto corriendo `php artisan migrate` (las migraciones del día no se habían aplicado). Al reintentar, apareció un segundo error distinto: `SQLSTATE[23000]: Integrity constraint violation: 19 CHECK constraint failed: status`, ya con `php artisan migrate:status` confirmando que las cuatro migraciones relevantes (incluida la que agrega `In Review/Verified by Registro` al enum) figuraban como "Ran".

2. Qué encontró la IA antes de actuar

Revisando las dos migraciones que tocan el enum de status (2026_08_21_194439 y 2026_08_22_190000), confirmó que ambas usan SQL crudo (`ALTER TABLE requests MODIFY status ENUM(...)`) sintaxis exclusiva de MySQL envuelto en `if (driver !== 'sqlite') { ... }`. Ese guard, agregado en una sesión anterior precisamente para no romper con un error de sintaxis en SQLite, tuvo un efecto secundario no previsto entonces: en SQLite el `ALTER` simplemente nunca se ejecuta, así que la restricción `CHECK` que SQLite usa para emular `enum()` se quedó exactamente como la dejó la migración de creación de la tabla (2026_08_07_100008), sin `Verified by Registro` (y, en su momento, sin `In Review` una vez que se había quitado). Es decir: las migraciones "corrieron" (aparecen como Ran) pero en SQLite no cambiaron nada.

3. Qué se aceptó de la respuesta de la IA

Una migración nueva (no se editaron las dos anteriores, ya aplicadas) que usa `Blueprint::enum('status', [...])->change()` — la API fluida de Laravel, que el propio framework traduce correctamente por motor (en MySQL genera el `ALTER MODIFY` equivalente; en SQLite reconstruye la tabla completa con la nueva restricción `CHECK`) en vez de seguir escribiendo SQL crudo por motor. Reemplaza definitivamente el patrón de "SQL crudo + guard por driver" que había quedado corto.

4. Qué se rechazó y por qué

No se editaron las migraciones 2026_08_21_194439 ni 2026_08_22_190000 para "arreglarlas" ya están marcadas como ejecutadas en las bases de datos existentes (la del equipo incluida), así que modificarlas no las volvería a correr; el único camino correcto en Laravel es una migración nueva que corrija el estado actual, no una edición retroactiva de una migración ya aplicada.

5. Qué hubo que corregir o verificar manualmente

php -l sobre la migración nueva. No fue posible ejecutar php artisan migrate en este entorno (sin vendor/) para confirmar que Blueprint::change() efectivamente reconstruye la tabla en SQLite sin requerir doctrine/dbal (no está en composer.json) — quedó pendiente que el equipo la corra y confirme, ya que es la única verificación real posible con la versión de Laravel de este proyecto.

6. Qué se aprendió del proceso

Un guard if (driver !== 'sqlite') alrededor de SQL crudo evita el error de sintaxis inmediato, pero no es lo mismo que "la migración funciona en SQLite" — sencillamente no hace nada ahí, y ese vacío solo se manifiesta más tarde, en el primer intento de escribir un valor que el guard nunca llegó a habilitar. La lección de fondo, ya aprendida una vez con el bug del MODIFY en la entrada del 21 de agosto y repetida aquí: cuando una migración necesita tocar un enum/CHECK constraint de forma que funcione en más de un motor de base de datos, la API fluida de Laravel (change()) es la herramienta correcta desde el principio.

Entrada — 25 de agosto de 2026 (correo real de demo y cambio del disparador de notificación de ES-03)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Configurar el envío real de correo (SMTP de Gmail) para una demo en vivo, y cambiar cuándo se dispara la notificación de ES-03: de "en cada cambio de estado" a "solo al enviar la solicitud", como confirmación de recepción

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la base de datos real (MySQL) y al servidor SMTP para pruebas de envíos reales

1. Qué se le consultó a la IA

En una sesión con varios pasos sucesivos: (1) insertar el correo real del usuario en la base de datos para que la notificación de una demo en vivo llegara a su bandeja; (2) configurar .env para que el correo saliera de verdad (no solo quedara en el log); (3) tras probarlo, reportar que enviar una solicitud de convalidación no generaba ninguna notificación; (4) pedir explícitamente que el correo notifique solo en el momento en que el estudiante envía la solicitud como confirmación de que se recibió con éxito y no en cada cambio de estado como estaba diseñado originalmente; (5) implementar ese cambio directamente y documentar la sesión en este diario.

2. Qué encontró la IA antes de actuar

1. Antes de tocar la base de datos, revisó el código de EloquentRequestNotifier y confirmó que el destinatario del correo es \$student->user->email es decir, el mismo campo users.email que Fortify usa como identificador de login. No existe un campo separado para "correo de notificación". Esto se documentó explícitamente al usuario antes de escribir el UPDATE, aunque su efecto colateral (invalidar la credencial de login vieja) solo se hizo evidente después, cuando el usuario reportó no poder entrar con estudianteISW@gmail.com.

2. Verificó contra la base de datos real (no contra TestDataSeeder.php, que resultó desactualizado) cuál era el usuario "estudiante" real vinculado a un StudentModel: el seeder de fixtures busca un usuario estudiante@gmail.com que no existe en esta base de datos (el real es estudianteISW@gmail.com, id 3), por lo que ese bloque de TestDataSeeder::seedWaiverEngineFixtures() nunca se ejecuta contra los datos actuales hallazgo colateral, fuera de alcance de esta sesión, no corregido aquí.

3. Antes de asumir que la falta de notificación al convalidar era un bug, leyó ChangeRequestStatusUseCase y CreateRequestUseCase: confirmó que notifyStatusChanged() solo se invocaba desde el primero, nunca desde el

segundo. Es decir, el comportamiento reportado por el usuario era el diseño original documentado en la entrada del 14 de agosto (ES-03 dice literalmente "en cada cambio de estado"), no un defecto.

4. Detectó, revisando .env, que MAIL_MAILER=log y QUEUE_CONNECTION=database con esa configuración ningún correo real habría salido nunca, sin importar qué tan bien estuviera la lógica de negocio. Lo señaló de forma proactiva, antes de que fallara en vivo frente al docente.

5. Ante la petición de "solo notificar al enviar", señaló explícitamente la tensión con el texto literal de ES-03 ("en cada cambio de estado") y presentó dos caminos: agregar la confirmación de envío sin quitar la de cambio de estado (más fiel al spec) o reemplazar una por la otra. No decidió por su cuenta cuál aplicar.

3. Qué se aceptó de la respuesta de la IA

- Actualizar el email del usuario de prueba (estudiantelSW@gmail.com, id 3) directamente en la base de datos vía php artisan tinker un cambio de dato, no de código fuente del proyecto, por lo que no requería la autorización especial de [[feedback_workflow]].
- La configuración de .env (MAIL_MAILER=smtp, host/puerto de Gmail, QUEUE_CONNECTION=sync para envío inmediato en la demo), incluyendo la contraseña de aplicación de Gmail proporcionada directamente por el usuario.
- La opción "reemplazar", elegida explícitamente por el equipo tras ver el trade-off planteado por la IA: se prioriza la confirmación de recepción para el estudiante sobre el texto literal de ES-03. Queda registrado aquí como desviación consciente del spec, no como un olvido, para que el equipo pueda defenderla en la exposición oral si el docente pregunta por qué no hay correo en los cambios de estado.
- El código de los 5 archivos modificados (RequestNotifierInterface, la nueva RequestSubmittedNotification, EloquentRequestNotifier, CreateRequestUseCase, ChangeRequestStatusUseCase), con autorización explícita del equipo desviación puntual de la regla por defecto de [[feedback_workflow]], igual que las autorizadas en sesiones anteriores.
- Eliminar por completo la clase RequestStatusChangedNotification.php en vez de dejarla sin usar, una vez confirmado (por búsqueda en todo el repo) que ningún otro archivo la referenciaba.

4. Qué se rechazó y por qué

- Se rechazó la opción que la IA señaló como más fiel al spec (agregar sin quitar) decisión de negocio del equipo, no un error de la IA.

- Se rechazó dar por buena la primera prueba de correo real solo porque "llegó" el equipo revisó el contenido real recibido y encontró que estaba en inglés pese a que el proyecto está configurado en español (APP_LOCALE=es), y pidió corregirlo antes de aceptar el cambio como terminado.

5. Qué hubo que corregir o verificar manualmente — el error real de la IA

El primer correo de prueba (RequestSubmittedNotification) llegó a la bandeja real del usuario en inglés, aunque el proyecto usa lang/es.json para traducir todos los textos vía __() y ya tenía traducciones para los mensajes equivalentes de la notificación vieja. La IA escribió los strings literales nuevos ("We received your :type", "We successfully received your :type for :course.", "Current status: :status") sin agregar sus entradas correspondientes a lang/es.json antes de probar un descuido real, detectado por el usuario al leer el correo recibido, no por la IA de forma proactiva antes de enviarlo.

Se corrigió agregando las 3 claves nuevas a lang/es.json y, en el mismo cambio, eliminando las 5 claves que quedaron huérfanas tras borrar

RequestStatusChangedNotification (Your :type is now :status, The status of your :type for :course has changed., Previous status: :status, New status: :status, Estimated resolution date: :date) verificado con una búsqueda en todo el repositorio, antes de borrarlas, de que ningún otro archivo PHP las usaba.

También se verificó, con datos reales y no solo lectura de código:

- Dos envíos de prueba end-to-end contra el SMTP real de Gmail (vía php artisan tinker llamando directamente a CreateRequestUseCase), el primero confirmando el bug de idioma y el segundo confirmando la corrección el usuario leyó el contenido real de ambos correos recibidos antes de aceptar el cambio.
- php artisan test (54/56) tras el cambio, y con git log sobre los 2 archivos de las pruebas que fallan, que ninguno de los dos fue tocado en esta sesión — confirmando que las fallas son preexistentes y no una regresión introducida aquí.
- Se limpiaron (DELETE) las 2 solicitudes de prueba creadas para disparar los correos, para no dejar datos de prueba visibles en la bandeja de Docencia durante la demo real frente al docente.

6. Qué se aprendió del proceso

- Cuando un mismo campo de base de datos cumple dos roles (aquí, users.email como login *y* como destinatario de notificaciones), cambiarlo para un propósito tiene un efecto colateral silencioso en el otro. Vale la pena,

la próxima vez, señalar *ambos* usos del campo *antes* de escribir el UPDATE, no solo el que motivó el cambio se habría evitado que el usuario quedara bloqueado tratando de iniciar sesión con la credencial vieja.

- Agregar una clase de notificación nueva con strings en inglés no es suficiente en un proyecto con archivo de traducciones central (lang/es.json): cada string literal nuevo pasado a ____() necesita su propia entrada, y la única forma confiable de comprobar que no falta ninguna es leer el correo real recibido, no solo que el código "compila" o que el envío no lanzó una excepción.
- Frente a una petición que se aleja del texto literal del spec (ES-03: "en cada cambio de estado"), señalar la tensión y las dos alternativas sin aplicar ninguna hasta que el equipo elija permitió que la decisión quedara documentada como consciente, con su razón, en vez de aparecer como una desviación no explicada si el docente la nota en la defensa oral.
- Verificar un cambio de notificaciones por correo enviando correos reales (SMTP real, bandeja real) en vez de Mail::fake() fue lo que expuso tanto el bug de idioma como la confirmación final para este tipo de cambio, la prueba automatizada no sustituye leer el correo tal como lo recibiría el usuario final.

Entrada — 25 de agosto de 2026 (continuación: el contador de Aprobado/Reprobado/Acreditadas/Créditos no se actualizaba al aprobar una solicitud)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Diagnóstico y corrección de que el resumen del expediente del estudiante (Aprobado/Reprobado/Acreditadas y Créditos), visible al final del detalle de una solicitud en el perfil de Docencia, nunca cambiaba después de que Docencia aprobara una solicitud

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la base de datos real para pruebas

1. Qué se le consultó a la IA

Se reportó, de forma coloquial: en el perfil de Docencia, al final del detalle de una solicitud, en la sección de "Aprobado/Reprobado/Acreditadas y Créditos", el contador debería cambiar después de que Docencia cambie el estado de la solicitud debería "registrar" ese cambio.

2. Qué encontró la IA antes de actuar

1. Localizó el panel descrito (request-component.blade.php:593-597) y confirmó que sí es dinámico: RequestComponent::studentRecord() lee en vivo la tabla academic_records del estudiante y calcula los conteos de aprobado/reprobado/acreditado y créditos ganados/totales a partir de esas filas no es un valor estático.

2. Antes de asumir que el bug estaba en el cálculo del resumen, revisó qué escribe en academic_records cuando Docencia aprueba una solicitud: nada. Ni ChangeRequestStatusUseCase ni ningún otro caso de uso tocan esa tabla.

3. Encontró que los estados 'Credited by Validation' y 'Requirement Waived' ya existen en el enum de la migración de academic_records (2026_08_07_100007_create_academic_records_table.php) y ya son reconocidos tanto por el resumen (RequestComponent::PROGRESS_STATUSES/CREDITED_STATUSES) como por el propio motor de reglas de ES-01 (EloquentStudentAcademicProfileRepository::PROGRESS_STATUSES, usado por countApprovedCourses()) es decir, el sistema ya sabe leer e interpretar esos dos estados como progreso académico, pero ningún código los produce. Confirmó, con una consulta directa a la base de datos antes de escribir nada, que ninguna fila de academic_records con esos dos estados existía en la base real.

4. Concluyó que se trata de una funcionalidad nunca conectada (mismo patrón que el motor de reglas de ES-01 antes de la entrada del 14 de agosto), no un bug de cálculo y lo presentó como tal antes de proponer una corrección.

3. Qué se aceptó de la respuesta de la IA

- El diagnóstico completo antes de tocar código.
- El diseño del puerto nuevo (AcademicRecordRegistrarInterface::registerCredit(Request \$request)) y su adaptador Eloquent (EloquentAcademicRecordRegistrar), siguiendo el mismo patrón Hexagonal ya establecido para RequestNotifierInterface/EloquentRequestNotifier Domain solo conoce la interfaz y la entidad Request, la traducción a un estado concreto de academic_records vive en Infrastructure.
- La regla de mapeo: 'Requirement Waiver' → 'Requirement Waived'; 'Validation' → 'Credited by Validation', aplicada sobre courseId() de la solicitud (el curso objetivo, no el prerrequisito) y usando updateOrCreate por (student_id, course_id) para no duplicar filas si una solicitud posterior vuelve a tocar el mismo curso.
- Enlazar el equivalence_id de la nueva fila de academic_records al id de la propia solicitud que originó el crédito no existía ningún otro dato con qué llenar esa columna (comentada en su migración como "Reference resolution for the credit"), y deja trazabilidad de qué solicitud generó cada crédito.
- Disparar el registro únicamente cuando \$newStatus === 'Approved', apoyándose en el invariante ya existente de Request::changeStatus() (un estado Approved es final e irreversible) para no necesitar ninguna comprobación adicional contra el estado anterior.
- El código de los 4 archivos (interfaz, adaptador, binding en DomainServiceProvider, y la llamada en ChangeRequestStatusUseCase), con autorización explícita.

4. Qué se rechazó y por qué

- No se propuso ni se aplicó ningún cambio a requiredCourseId de la solicitud de tipo Requirement Waiver el usuario solo pidió que el contador reflejara el cambio de estado de la solicitud aprobada, no rediseñar qué pasa con el curso prerrequisito, que ya debía tener su propio registro de "Aprobado" desde antes.

5. Qué hubo que corregir o verificar manualmente

- Se probaron ambos flujos de punta a punta contra la base de datos real (no solo lectura de código): crear una solicitud de Convalidación → aprobarla como Docencia → confirmar que apareció una fila academic_records con

status = 'Credited by Validation'; y lo mismo para una solicitud de Levantamiento de Requisito, confirmando status = 'Requirement Waived'. Ambas pruebas se limpiaron después (DELETE) para no dejar datos de prueba en la base real.

- Durante la segunda prueba (Levantamiento de Requisito) apareció un error de base de datos (Data truncated for column 'waiver_justification') al usar un valor de texto libre para ese campo la IA verificó con SHOW COLUMNS que la columna es en realidad un ENUM con 5 valores fijos predefinidos, no texto libre. Esto no era un bug del cambio de esta sesión, sino un error de la propia IA al construir el dato de prueba sin revisar antes el esquema real de la columna; se corrigió usando uno de los valores válidos del enum y la prueba pasó.
- Se corrió php artisan test (54/56) después del cambio: mismas 2 fallas preexistentes de sesiones anteriores (confirmadas por git log como no tocadas en esta sesión), sin regresiones nuevas.

6. Qué se aprendió del proceso

- Un valor de enum reconocido y consumido en varios lugares del código (PROGRESS_STATUSES en dos clases distintas) pero nunca producido por ningún caso de uso es una señal fuerte de funcionalidad a medio conectar — vale la pena, ante un reporte de "esto no se actualiza", buscar primero quién *lee* el dato (para confirmar que el cálculo en sí está bien) y después quién *escribe* el dato (que es donde suele estar el hueco real), en ese orden.
- Cuando dos casos de uso ya comparten un patrón (aquí, ChangeRequestStatusUseCase ya inyectaba un puerto de notificación siguiendo Hexagonal), replicar exactamente ese mismo patrón para una necesidad nueva relacionada mantiene el código predecible no hubo que inventar una convención nueva para esta funcionalidad.
- Antes de escribir un dato de prueba contra una columna desconocida, revisar su definición real (SHOW COLUMNS) evita perder tiempo diagnosticando un error de base de datos que en realidad era un dato de prueba inválido, no un bug del cambio que se estaba verificando.

Entrada — 25 de agosto de 2026 (rol RBAC "Registro")

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Creación del rol y usuario de prueba "Registro", con los mismos permisos que "Coordinadora de Docencia" tiene hoy

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la base de datos real

1. Qué se le consultó a la IA

Se pidió crear un usuario "Registro" con las capacidades actuales de Docencia, y verificar explícitamente que ese usuario fuera capaz de cambiar el estado de una solicitud a "Aprobada".

2. Qué encontró la IA antes de actuar

Ante la ambigüedad de si "usuario de Registro" significaba un rol RBAC nuevo o solo un login adicional bajo el rol Docencia ya existente, la IA no asumió ninguna de las dos se lo preguntó directamente al equipo, señalando que el estado 'Verified by Registro' ya existente en el pipeline de solicitudes (desde la entrada del 22 de agosto) sugería que "Registro" ya era, de hecho, un actor previsto en el diseño, aunque nunca se le hubiera dado un rol propio.

3. Qué se aceptó de la respuesta de la IA

- La opción de crear un rol RBAC nuevo "Registro" (no solo un usuario bajo el rol Docencia), con el mismo conjunto de 13 permisos que Docencia tiene hoy decisión explícita del equipo tras la pregunta de la IA, no una elegida unilateralmente.
- El usuario de prueba registro@gmail.com / 12345678, siguiendo el mismo patrón de credenciales que los demás usuarios de DatabaseSeeder.php.
- La verificación explícita, contra la base de datos real (no solo lectura de código), de que el nuevo usuario sí puede cambiar el estado de una solicitud a "Aprobada" se confirmó con RequestPolicy::review(), que autoriza por el permiso requests.review y no por nombre de rol, así que cualquier rol con ese permiso queda habilitado automáticamente.

5. Qué hubo que corregir o verificar manualmente

Ninguna corrección se verificó de punta a punta (crear solicitud → aprobar como Registro → confirmar reviewer_id correcto) contra la base de datos real antes de dar el cambio por terminado.

6. Qué se aprendió del proceso

- Un estado ya presente en un enum ('Verified by Registro') puede ser evidencia de una decisión de diseño tomada pero nunca completada (ningún

rol tenía permiso para usarlo hasta ahora) vale la pena, al encontrar un valor así, preguntarse qué actor lo iba a usar antes de asumir que es solo un nombre descriptivo sin consecuencia de permisos.

- Cuando una instrucción del usuario es ambigua entre dos implementaciones con costos muy distintos (un rol RBAC nuevo vs. reutilizar uno existente), preguntar antes de elegir evita tener que deshacer trabajo si la lectura equivocada resulta ser la incorrecta.

Entrada — 25 de agosto de 2026 (estado de aprobación en dos etapas:
Docencia → Registro)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: División del estado único "Aprobada"/"Denegada" en dos etapas la decisión sustantiva de Docencia y el cierre final, exclusivo de Registro

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la base de datos real para migrar datos existentes

1. Qué se le consultó a la IA

Se pidió, en una sola instrucción: renombrar el estado "Aprobada" a "Aprobada por Docencia" ("y así sucesivamente"), y que la aprobación *final* de una solicitud fuera un estado nuevo, "Aprobada por Registro", que solo el rol Registro pudiera aplicar.

2. Qué encontró la IA antes de actuar

1. Antes de tocar nada, verificó cuántas solicitudes reales ya estaban en estado 'Approved' en la base de datos: 6, todas creadas ese mismo día probando la demo dato crítico porque cualquier cambio de enum sin migrar esas filas las habría dejado en un estado inválido.

2. Identificó, revisando RequestPolicy, que Docencia y Registro comparten hoy exactamente los mismos permisos (requests.review) por lo tanto, la restricción "solo Registro puede dar la aprobación final" pedida por el equipo no existía todavía como regla de autorización; hacía falta una nueva, no bastaba con el permiso genérico ya usado para todo lo demás.

3. Localizó, con una búsqueda de los literales 'Approved'/'Denied' en todo el repositorio, exactamente qué archivos pertenecían al estado de requests.status (a renombrar) frente a cuáles pertenecían a enums no relacionados que usan las mismas palabras validation_precedents.result (el resultado de un precedente histórico de convalidación) y academic_records.status (que sí incluye 'Approved'/'Failed' pero es un concepto distinto, el expediente académico). Confirmó que ninguno de esos dos debía tocarse, evitando renombrar un enum equivocado.

4. Revisó las tres migraciones previas que ya habían tocado esta misma columna (2026_08_21_194439, 2026_08_22_190000, 2026_08_23_000000_fix_status_enum_on_sqlite) para entender el patrón ya aprendido por el equipo: alterar un ENUM/CHECK en más de un motor de base de datos debe hacerse con Blueprint::change(), nunca con SQL crudo por driver

y que remapear datos existentes *antes* de estrechar el enum es obligatorio para no perderlos.

5. Detectó que remapear datos a los 4 valores nuevos requería primero ampliar el enum (para que esos valores nuevos fueran válidos) y solo después angostarlo quitando los dos valores viejos hacerlo al revés habría repetido el mismo error de truncado de datos ya visto en una sesión anterior con la columna `waiver_justification`.

3. Qué se aceptó de la respuesta de la IA

- Los nombres literales elegidos para los 4 estados nuevos ('Approved by Docencia', 'Denied by Docencia', 'Approved by Registro', 'Denied by Registro'), siguiendo el mismo patrón ya usado por 'Verified by Registro' (usar el nombre del rol tal cual, no traducirlo a "Registrar's Office").
- El nuevo permiso `requests.finalize`, asignado solo al rol Registro (nunca a Docencia), como el mecanismo de autorización que hacía falta para la restricción pedida siguiendo el mismo patrón ya establecido para `requests.review` (un permiso nombrado, no una condición de nombre de rol embebida en la política).
- Migrar automáticamente las 6 solicitudes ya 'Approved' a 'Approved by Docencia' (no directo a 'Approved by Registro') decisión explícita del equipo ante la pregunta de la IA, consistente con que ese paso de Registro no existía cuando esas solicitudes se aprobaron.
- Que 'Denied' siguiera el mismo patrón de dos etapas que 'Approved' también decisión explícita del equipo ante la pregunta de la IA, no asumida por simetría.
- Tres decisiones de diseño que la IA tomó sin preguntar, presentadas después como parte del resumen para que el equipo las revisara: (a) la detección de solicitudes duplicadas de un levantamiento bloquea ya desde 'Approved by Docencia', no solo desde el cierre final de Registro, para no dejar que un estudiante reenvíe la misma solicitud mientras espera el cierre administrativo; (b) el indicador "Reconocido"/"No reconocido" de Convalidación se ilumina ya en la etapa de Docencia, no hasta que Registro cierre; (c) el "stepper" de progreso solo marca la solicitud como resuelta en la etapa final de Registro.

4. Qué se rechazó y por qué

No se rechazó ninguna propuesta de la IA en esta sesión las tres preguntas de alcance planteadas antes de escribir código (qué pasa con las filas existentes, si Denegada también se parte en dos, si la restricción de autorización debía ser

nueva) se resolvieron todas a favor de la opción que la IA marcó como recomendada.

5. Qué hubo que corregir o verificar manualmente

No hubo errores de la IA que corregir en esta sesión, pero sí una verificación en vivo antes de dar el cambio por terminado, dado que se trata de la máquina de estados central del módulo:

- Migración de datos ejecutada contra la base de datos real: confirmado con SHOW COLUMNS que el nuevo ENUM quedó con los 7 valores esperados, y con un GROUP BY status que las 6 filas 'Approved' pasaron a 'Approved by Docencia' sin perder ninguna.
- Prueba de autorización en vivo con Gate::forUser(): confirmado que Docencia NO puede aplicar 'Approved by Registro' (se le niega el permiso finalize) y que Registro sí puede.
- Dos flujos completos de punta a punta contra la base de datos real: un Levantamiento de Requisito (crear → Docencia aprueba → intento de duplicado bloqueado correctamente → Registro cierra → fila de academic_records creada) y una Convalidación vía "Reconocer" (mismo recorrido) confirmando que el registro de crédito académico (de la sesión anterior) sigue disparándose correctamente, ahora en el momento correcto ('Approved by Registro', no 'Approved by Docencia').
- Se actualizaron los literales de estado en RequestTest.php (suite de dominio) y en RequestFactory::automaticallyApproved() para que siguieran probando el invariante real; php artisan test se corrió después: 54/56, mismas 2 fallas preexistentes de sesiones anteriores, sin regresiones nuevas.
- Datos de prueba de las verificaciones en vivo eliminados (DELETE) al terminar, para no dejar residuos en la bandeja de Docencia real.

6. Qué se aprendió del proceso

- Antes de renombrar un valor de enum, buscarlo en *todo* el repositorio y clasificar cada aparición por a qué tabla pertenece realmente evita el error de tocar un enum de apariencia idéntica pero significado distinto (aquí, tres enums distintos comparten los literales 'Approved'/'Denied': requests.status, validation_precedents.result, academic_records.status).
- Cuando un cambio de estado necesita una regla de autorización más fina que la ya existente ("todo el que puede revisar, puede aprobar" → "solo un subconjunto puede cerrar"), el patrón correcto en este proyecto es un permiso nuevo y nombrado (requests.finalize), no una condición de nombre de rol dentro de la política mantiene la autorización basada en permisos, consistente con el resto del sistema.

- Una migración de ENUM que a la vez agrega valores nuevos y quita valores viejos no puede hacerse en un solo ALTER: hay que ampliar primero (para que el UPDATE de los datos existentes sea válido contra la restricción), remapear los datos, y angostar después el orden importa tanto como en el bug de waiver_justification de una sesión anterior, y es la misma lección aplicada a un caso más grande.
- En un cambio que toca la máquina de estados central de un módulo (no una funcionalidad aislada), vale la pena verificar cada arista del comportamiento nuevo por separado contra datos reales (autorización, transición, efectos secundarios como el registro de crédito) en lugar de solo confirmar que "la suite de tests sigue en verde" los tests existentes no cubrían este flujo nuevo, así que pasar en verde no habría sido evidencia suficiente por sí sola.

Entrada — 25 de agosto de 2026 (ajustes de UI sobre el flujo Docencia/Registro)
Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Tres ajustes puntuales pedidos justo después de implementar el flujo en dos etapas: separar los botones de estado por rol, un check visual de confirmación al guardar datos externos, y la vista previa de documentos como ventana emergente

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal

1. Qué se le consultó a la IA

Tres pedidos sucesivos, cada uno tras revisar en el navegador el resultado del cambio anterior: (1) el equipo notó, viendo la pantalla como Docencia, que el botón "Verificada por Registro" seguía apareciendo ahí aunque ese estado es exclusivo de Registro pidió quitarlo; luego, viendo la pantalla como Registro, notó que también aparecían los botones de Docencia ("Pendiente de revisión", "Aprobada por Docencia", etc.) y pidió que cada perfil viera solo lo que le corresponde; (2) tras preguntar qué hacía el botón "Guardar datos externos", pidió agregar un check verde de "guardado correctamente"; (3) pidió que la vista previa de un documento adjunto abriera en una ventana emergente en lugar de como estaba.

2. Qué encontró la IA antes de actuar

1. Para el primer pedido, confirmó que el permiso `requests.finalize` (creado en la entrada anterior) ya era exactamente el mecanismo necesario para distinguir ambos conjuntos de botones no hacía falta ningún permiso nuevo, solo cambiar la lista de estados de "unión" (base + extra si puede finalizar) a "exclusión mutua" (uno u otro conjunto completo según `canFinalize`). Señaló proactivamente, sin que se preguntara, dos efectos colaterales de este cambio: Registro conserva el permiso `requests.review` además de `finalize`, así que técnicamente aún podría alcanzar los estados de Docencia si alguien forzara la petición directamente (el cambio aplicado es solo de interfaz, no una restricción nueva de backend); y Superadmin, al pasar todas las validaciones de permisos automáticamente, ahora solo vería los 3 botones de Registro en vez del set completo. Ninguno de los dos se resolvió sin preguntar quedaron señalados para que el equipo decida si hace falta ajustarlos.

2. Para el segundo pedido, revisó que el proyecto ya tenía una convención visual establecida (`.form-success`, usada para "Archivo adjuntado" en los campos de carga de documentos) y la reutilizó en vez de inventar un estilo nuevo.

3. Para el tercero, confirmó que solo existía un lugar en todo el proyecto con el enlace de vista previa (`request-component.blade.php`), y que ya usaba Alpine.js

en el mismo archivo (@click en el acordeón del expediente académico) así que implementó el popup con @click.prevent + window.open() en vez de introducir una librería nueva.

3. Qué se aceptó de la respuesta de la IA

- La separación de los botones de estado en dos conjuntos mutuamente excluyentes (Docencia vs. Registro) según \$viewingRequest['canFinalize'].
- El check verde reutilizando .form-success, con una propiedad Livewire nueva (\$externalCourseDataSaved) que se resetea automáticamente si el código o los créditos externos se vuelven a editar, para que el check nunca quede mostrando "guardado" sobre datos sin guardar.
- La vista previa como ventana emergente de 900×750px vía window.open(), conservando el href del enlace para que un clic central o "abrir en pestaña nueva" manual del usuario siga funcionando.

4. Qué se rechazó y por qué

Ninguno de los tres cambios fue rechazado ni corregido el equipo los aceptó tal como se propusieron.

5. Qué hubo que corregir o verificar manualmente

- php artisan test corrido después de cada uno de los tres cambios (54/56, mismas 2 fallas preexistentes) pero la IA fue explícita en que esto no prueba nada de los cambios en sí: los tres son puramente de interfaz (Blade/Alpine/Livewire sin lógica de dominio nueva), así que la suite automatizada no los ejerce; la verificación real queda pendiente de que el equipo los pruebe visualmente en el navegador.

6. Qué se aprendió del proceso

- Cuando un permiso ya fue diseñado para distinguir dos roles (aquí, requests.finalize), suele bastar para resolver pedidos de UI relacionados sin tener que tocar la capa de autorización de nuevo el trabajo de diseño de la entrada anterior se pagó solo en esta.
- Un cambio de interfaz "que solo oculta botones" puede dar una falsa sensación de restricción completa si el permiso subyacente sigue siendo más amplio que lo que la UI ahora muestra vale la pena decirlo explícitamente en vez de dejar que el equipo asuma que ocultar el botón es lo mismo que bloquear la acción.
- Tres cambios pequeños y sucesivos, cada uno verificado brevemente antes de pasar al siguiente, permitieron detectar rápido cuando algo no coincidía con lo pedido (como en el mensaje anterior, donde el equipo pidió quitar los botones de Docencia del perfil de Registro después de ver el resultado del

primer ajuste) iterar en pasos cortos con revisión visual entre cada uno resultó más eficiente que intentar adivinar el diseño final de una sola vez.

Entrada — 25 de agosto de 2026 (correos de confirmación, validaciones en tiempo real y ajustes UX del módulo de Solicitudes)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Serie de correcciones y mejoras sobre el flujo de Solicitudes del estudiante: reestructuración del correo de confirmación, dos bugs de validación de negocio, un diálogo de confirmación de envío, cambios de UX en “Mis solicitudes” y en el detalle de la solicitud, y consolidación del correo de Convalidación en un solo envío.

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la base de datos real, a `php artisan tinker/Livewire::test()` para pruebas de extremo a extremo, y al navegador (Chrome) para verificación visual en vivo.

1. Qué se le consultó a la IA

- Levantar el entorno local (servidor, migraciones) y limpiar caché de Laravel.
- Corregir un bug de negocio: al crear una solicitud de Levantamiento de requisitos, el curso “Requisito no cumplido” podía quedar igual al “Curso a matricular”; y que el error apareciera apenas se seleccionara, no solo al enviar.
- Rediseñar por completo el correo de confirmación de envío (mostrado primero como borrador de texto antes de tocar código): agregar el nombre del documento adjunto y la fecha de envío, quitar la fecha estimada, y corregir el orden real de revisión (Docencia primero, luego Registro no al revés, como decía el borrador de referencia que trajo el usuario).
- Diagnosticar por qué el envío de una solicitud se sentía lento.
- En “Mis solicitudes”: agregar un buscador que filtre las 4 columnas visibles, y cambiar la columna “Fecha Estimada” por “Fecha de envío”.
- Corregir la etiqueta “Dispensa de requisito” a “Levantamiento de requisitos” en toda la aplicación.
- Agregar un diálogo “¿Está seguro?” antes de procesar cualquier envío de solicitud, para prevenir errores de usuario con una segunda ronda para simplificar el texto exacto.
- Corregir un segundo bug de negocio: en Convalidación, el formulario permitía seleccionar el mismo curso de la UTN dos veces entre varias filas; y que el error se limpiara automáticamente al borrar la fila duplicada, no solo al cambiar la selección.

- En el detalle de una solicitud: usar la etiqueta correcta “Requisito no cumplido”, quitar los campos “Resultado” y “Fecha Estimada” (redundantes con “Estado”), y agregar “Fecha de envío”.
- Rediseñar el detalle de Convalidación como una tabla de 3 columnas (Curso UTN / Curso externo / Universidad) primero pensada para agrupar todos los cursos del mismo envío, y luego, tras aclarar el equipo que cada curso de una Convalidación se revisa de forma independiente, revertida a mostrar solo la fila propia de esa solicitud.
- Consolidar el correo de Convalidación: si un estudiante convalida varios cursos en un mismo envío, debe llegar un solo correo listando todos los cursos, no uno por curso.

2. Qué encontró la IA antes de actuar

- El correo se sentía lento no por un problema en el código de la notificación (que ya implementaba ShouldQueue correctamente) sino porque `QUEUE_CONNECTION=sync` en él `.env` hacía que Laravel ejecutara el job de forma síncrona, dentro de la misma petición HTTP que el estudiante esperaba la IA identificó la causa raíz antes de proponer cambiar a `QUEUE_CONNECTION=database` más un proceso `queue:work` aparte.
- La etiqueta “Requisito no cumplido” no se corregía con un cambio simple porque la clave de traducción “Unmet requirement” estaba duplicada dos veces en `lang/es.json`, con valores distintos (“Requisito no cumplido” y “Requisito incumplido”) un archivo JSON no admite claves duplicadas de verdad; PHP simplemente se queda con la última al decodificarlo, que era la incorrecta.
- La validación en tiempo real del curso duplicado en Convalidación no funcionaba pese a tener la regla `distinct` correctamente declarada la IA revisó el código fuente de Livewire (`HandlesValidation::filterCollectionDataDownToSpecificKeys()`) y confirmó que `Form::validateOnly()` aísla deliberadamente el campo validado del resto de su array hermano antes de pasarlo al Validator, dejando la regla `distinct` sin nada contra qué comparar. La corrección real fue un chequeo manual de duplicados en el propio componente, no un ajuste a la regla.
- Al implementar el agrupamiento del detalle de Convalidación por lote de envío (`batch_id`), tras un intercambio con el equipo se aclaró que el modelo correcto es que cada curso de una Convalidación es una solicitud independiente con su propia revisión la IA revirtió la migración, la entidad de dominio, el DTO, el repositorio y el caso de uso agregados para esa función, en lugar de dejarlos sin usar.

- Un correo desactualizado a mitad de sesión: el worker de cola (php artisan queue:work) carga el código una sola vez al arrancar y lo mantiene en memoria para todos los jobs siguientes un cambio al contenido del correo no se reflejaba hasta reiniciar el proceso. La IA lo diagnosticó comparando el correo recibido contra lo acordado y la hora del último reinicio del worker, y desde entonces reinició el worker después de cada cambio a código relacionado con notificaciones.

3. Qué se aceptó de la respuesta de la IA

- El diagnóstico y la solución de la cola síncrona (cambiar a `QUEUE_CONNECTION=database` con un proceso `queue:work` corriendo aparte).
- La estructura completa del nuevo correo de confirmación, mostrada primero como texto plano para aprobación antes de escribir una sola línea de código, con dos rondas de ajuste (quitar fecha estimada, agregar documento adjunto) antes de implementarla.
- El uso de la regla `different:courseId` (Levantamiento) y `distinct` (Convalidación) de Laravel para los dos bugs de negocio, con mensajes de error personalizados en español.
- El diseño del diálogo de confirmación de envío, reutilizando el mismo componente visual ya usado para confirmaciones de borrado en el resto de la aplicación en vez de crear un estilo nuevo con el texto final simplificado a pedido del usuario (quitar la redundancia entre el título “¿Estás seguro?” y la pregunta debajo).
- El flag `notify` agregado a `RequestDTO` para que, de varias solicitudes creadas en un mismo envío de Convalidación, solo la primera dispare el correo la única con `batchCourseNames` visible en el cuerpo del mensaje.
- La corrección de la traducción duplicada “Unmet requirement” a una sola entrada correcta, con efecto retroactivo en toda la aplicación (bandeja de staff incluida, no solo la pantalla del estudiante).

4. Qué se rechazó y por qué

- El agrupamiento de varios cursos de Convalidación en una sola vista de detalle (`batch_id`) fue implementado, probado en el navegador, y luego revertido por completo a pedido del equipo: cada curso se revisa de forma independiente, así que el detalle de una solicitud individual solo debe mostrar su propio curso mostrar ahí los cursos hermanos daba la impresión incorrecta de que la revisión era conjunta.
- Se descartó dejar la infraestructura de `batch_id` sin usar “por si acaso” una vez revertida la función que la necesitaba se retiró la migración, la entidad, el DTO y el repositorio en el mismo cambio, para no dejar código muerto.

5. Qué hubo que corregir o verificar manualmente

- Cada bug de negocio y cada flujo de correo se probó de punta a punta contra la base de datos real, primero vía php artisan tinker y Livewire::test(), y al final directamente en el navegador (Chrome) con una convalidación real de varios cursos incluyendo revisar el log del worker de cola para confirmar que se procesó exactamente un job de notificación, no varios.
- Quedaron solicitudes de prueba en la base de datos real del estudiante (varias entradas de “Inglés I/II/III” y otro par de cursos, usadas para verificar el agrupamiento y su reversión) pendientes de que el equipo decida si se eliminan.
- Se corrigió manualmente un archivo PDF de prueba con tamaño 0 bytes (Livewire::test() vía tinker no genera un archivo temporal real como sí lo hace un TestCase de PHPUnit) que hacía fallar la restricción CHECK `chk_files_size_bytes` de la tabla files no era un bug del cambio en sí, sino una limitación del entorno de prueba improvisado.

6. Qué se aprendió del proceso

- Un síntoma de UX (“se siente lento”) puede tener una causa de configuración de infraestructura (QUEUE_CONNECTION) en vez de una de código de aplicación vale la pena revisar la configuración del entorno antes de optimizar el código de la función que parece lenta.
- Una clave de traducción duplicada en un archivo JSON no produce ningún error visible: el archivo sigue siendo JSON válido, y solo se manifiesta como “la traducción no cambia, aunque edite la línea correcta” hay que buscar todas las apariciones de la clave, no asumir que hay una sola.
- El comportamiento interno de un framework (aquí, cómo Livewire recorta los datos de un array antes de validar un campo específico) puede invalidar silenciosamente una regla de validación correcta en el código pero que nunca llega a evaluarse contra los datos que debería cuando una regla declarada correctamente “no hace nada” en tiempo real pero sí funciona en el envío completo, vale la pena sospechar de la ruta de validación parcial del framework, no de la regla en sí.
- Antes de construir una funcionalidad que agrupa datos relacionados (aquí, varias solicitudes del mismo envío), vale la pena confirmar con el equipo si el modelo de negocio real trata esos datos como una unidad o como entidades independientes construirlo primero y descubrir después que el modelo no correspondía significó revertir una migración completa, aunque se hizo sin dejar residuos.
- Un worker de cola en ejecución no es un proceso “gratis” de reiniciar mentalmente: seguirá sirviendo código viejo desde memoria hasta que se

reinicie explícitamente, y ese olvido puede parecer un bug del código nuevo cuando en realidad el código nuevo nunca llegó a ejecutarse.

Entrada — 26 de agosto de 2026 (cierre del módulo: resolución formal de Registro, accesibilidad por teclado, endurecimiento de la URL y limpieza de comentarios)

Contribuidor (commit de git): WGuillenDev (wilberthguillen.ing@gmail.com)

Tema: Sesión larga de cierre del módulo de Solicitudes Estudiantiles. Se construyó la emisión formal de la resolución por parte de Registro (RSREC-001 / SLR-002), se ajustó el seguimiento visible para el estudiante, se eliminó el correo de resolución, se corrigieron permisos del rol Admin, se endureció el parámetro de la URL contra manipulación, se reparó la accesibilidad por teclado de toda la interfaz y se hizo una revisión sistemática de los comentarios del código.

Herramienta consultada: Claude (Anthropic), vía Claude Code en terminal, con acceso a la base de datos real, a php artisan tinker para pruebas de extremo a extremo, y al navegador (Chrome) para verificación visual y de teclado en vivo.

1. Qué se le consultó a la IA

- Construir la emisión de la resolución formal por parte de Registro: un panel con número de resolución, sesión, acta y fecha, dos botones explícitos (Publicar y Aprobar / Publicar y Denegar) y un diálogo de advertencia previo.
- Usar los documentos institucionales reales aportados por el equipo (el PDF editable SLR-002 y el archivo RSREC-001) para generar el documento de resolución con los datos del sistema.
- Ampliar el rastreador "Avance de la solicitud" de 2 a 4 hitos en el perfil de staff, y agregar esos mismos hitos al historial de estados.
- Eliminar el correo de resolución: el estudiante consulta el resultado entrando al sistema.
- Hacer obligatorios, del lado de Registro, los cuatro campos de la resolución; y del lado de Docencia, los campos Código/Créditos/Nota y la decisión Reconocer/No reconocer antes de resolver, más la razón cuando se elige No reconocer.
- Mostrar siempre la fecha estimada de resolución (recepción + 24 h) tanto en Docencia como en Registro, conservando la posibilidad de modificarla.
- Agregar al perfil del estudiante un avance de 5 hitos como primer elemento del detalle, y simplificar la columna Estado a Pendiente de revisión / Aprobada / Denegada.

- Verificar que se notifica por correo al enviar una solicitud nueva.
- Verificar que los roles Admin y Superadmin funcionan correctamente.
- Impedir la manipulación de la dirección en la barra del navegador para evitar ataques.
- Verificar que la accesibilidad por teclado (recorrido con Tab) funcione correctamente.
- Revisar todo el código del proyecto y dejar solo comentarios necesarios y con estructura profesional.

2. Qué encontró la IA antes de actuar

- Sobre la petición de "no permitir editar la dirección del host": la IA rechazó la premisa. Ninguna aplicación web puede bloquear la barra de direcciones del navegador; es una función del navegador, no del sitio. En su lugar identificó el riesgo real: el parámetro `?tab=` estaba enlazado a una propiedad de Livewire sin validación, y un valor inventado caía en la rama por defecto de un match, que devolvía un arreglo de filtros vacío y mostraba la lista sin filtro de tipo. No era escalada de privilegios (la autorización y el filtro por rol seguían aplicándose), pero sí entrada de usuario sin validar.
- Al auditar la accesibilidad encontró que todo el topbar estaba construido con elementos div con manejadores de clic: el botón de menú, el selector de tamaño de fuente (la propia función de accesibilidad), el modo oscuro y el menú de usuario. Ninguno era alcanzable por teclado, lo que significaba que un usuario que solo usa teclado no podía cerrar sesión. Los tres encabezados desplegados del menú lateral tenían el mismo problema.
- También detectó que no existía ningún indicador de foco visible en botones ni enlaces (fallo del criterio WCAG 2.4.7), ni enlace para saltar al contenido, y que los modales carecían de `role="dialog"`, no movían el foco al abrirse, no lo atrapaban dentro y no se cerraban con Escape. En cambio, confirmó que los modales cerrados sí usaban `display:none`, por lo que nunca habían sido tabulables un punto que ya estaba correcto.
- Al revisar los roles encontró que el rol Admin tenía 36 de 37 permisos: le faltaba `requests.finalize`, agregado durante esta misma sesión. Peor aún, `RoleSeeder.php` nunca le asignaba ningún permiso, de modo que una reconstrucción de la base desde cero habría dejado a Admin sin acceso a nada. Superadmin no dependía de esto por el `bypass` de `Gate::before`.
- Al verificar el correo de solicitud nueva confirmó que el código era correcto, pero que no había ningún worker de cola en ejecución, por lo que los correos quedaban encolados indefinidamente. Además, encontró cuatro trabajos huérfanos de `ResolutionIssuedNotification` (la clase eliminada esa misma sesión) que fallaban al procesarse.

- Al revisar los comentarios midió una densidad del 20,5 % (2.479 de 12.074 líneas), con archivos entre 45 % y 67 %. Detectó que su primera métrica producía falsos positivos: archivos como Permission.php aparecían al 40 % pero eran solo anotaciones @property/@return que usan el IDE y el análisis estático, no prosa, y refinó la medición.
- Durante esa revisión encontró dos comentarios que habían quedado factualmente falsos: uno afirmaba que nada escribe historial al crear una solicitud (falso desde que se agregó el hito "Recibida por Docencia"), y otro decía que la constancia se enviaba por correo al estudiante (falso desde que se eliminó ese correo en esta misma sesión).
- Al no poder extraer texto legible del archivo RSREC-001 (un .doc binario), lo informó en vez de inventar el contenido, y propuso dos estrategias distintas según el tipo de solicitud.

3. Qué se aceptó de la respuesta de la IA

- La reinterpretación de la petición sobre la URL: validar el parámetro en el servidor con una lista blanca, normalizada al montar el componente y en cada actualización, y cambiar la rama por defecto para que caiga al filtro más restrictivo en lugar del más abierto.
- El llenado del formulario AcroForm real SLR-002 mediante un script de Python con pypdf, invocado desde PHP, dado que no había librería de llenado de formularios disponible en PHP; y la renderización de una plantilla Blade a PDF para las convalidaciones, al no poder llenarse el .doc oficial sin Word o LibreOffice en el servidor.
- La corrección de accesibilidad completa: conversión de los nueve controles a elementos button reales con sus atributos ARIA, indicador de foco con :focus-visible, enlace para saltar al contenido, y gestión de foco en los modales (entrada, atrapado, Escape y retorno al elemento que los abrió).
- La decisión de enfocar el contenedor del diálogo en lugar del primer control al abrir un modal, que además de ser la práctica recomendada corrigió un salto de scroll no deseado.
- La corrección del rol Admin en dos frentes: la base de datos actual y RoleSeeder.php, para que reciba automáticamente todos los permisos existentes y no vuelva a quedar desactualizado.
- La separación de la clave de error del campo "Razón" respecto de "Comentario adicional": ambos comparten la misma propiedad de PHP, y usar la misma clave marcaba como obligatorio un campo que no lo era.
- El método displayEstimatedResolutionDate() en la entidad de dominio, que calcula la fecha al vuelo sin persistir nada, para que el detalle nunca muestre un guion cuando la fecha ya es conocida.

- La simplificación del estado visible para el estudiante a tres valores, dejando intacto el estado real que necesita el rastreador de avance.

4. Qué se rechazó y por qué

- Se rechazó la primera implementación del panel de resolución, en la que la decisión de Registro se derivaba automáticamente del veredicto de Docencia con un solo botón. El equipo aclaró que Registro toma su propia decisión, independiente: se refactorizó a dos botones explícitos y un parámetro de decisión propio.
- Se rechazó bloquear la barra de direcciones tal como se pidió literalmente, por ser técnicamente imposible; se explicó por qué y se implementó la protección que sí corresponde.
- Se rechazó dejar en la cola los cuatro trabajos huérfanos de la notificación eliminada: se borraron junto con el registro de trabajos fallidos, en lugar de dejarlos fallando en cada intento.
- Se rechazó reescribir a ciegas los comentarios de los 183 archivos: se priorizó por densidad de prosa real y se dejó constancia de qué quedó sin revisar y por qué.
- Se rechazó tocar las anotaciones de tipo (@property, @return) pese a inflar la métrica, por ser información que consumen el IDE y el análisis estático.

5. Qué hubo que corregir o verificar manualmente

- Cada función se verificó en el navegador con datos reales creados y eliminados para la prueba: las tres etapas del rastreador de avance, el bloqueo de los cuatro campos obligatorios de Registro, el bloqueo de Reconocer/No reconocer sin datos del curso, la razón obligatoria al no reconocer, la fecha estimada en ambos perfiles, y los perfiles Admin y Superadmin.
- El correo de solicitud nueva se verificó enviando una solicitud real desde el formulario del estudiante, con archivo adjunto, comprobando que el trabajo se encolaba y que el worker lo procesaba con éxito contra el SMTP de Gmail.
- La accesibilidad se verificó pulsando teclas reales: Tab para recorrer, Enter para activar el selector de tamaño de fuente y el menú de usuario, y Escape para cerrar. Se confirmó que el foco vuelve exactamente al elemento que abrió cada modal.
- Los 183 archivos PHP se validaron sintácticamente después de cada tanda de cambios de comentarios, para descartar bloques de comentario cerrados por accidente.
- Algo pendiente de decisión del equipo es que: al reemplazar la regla de "5 días hábiles" por "recepción + 24 h", quedaron eliminados HolidayCalendarInterface y NagerDateHolidayCalendar, que consumían la

API pública Nager.Date. Esa era la única integración con una API REST externa del proyecto, y el documento oficial la exige de forma obligatoria (Sección 3.b).

- Quedaron en la base de datos real algunas solicitudes de prueba creadas y borradas durante la sesión; se eliminaron todas al terminar cada verificación, junto con los archivos que habían generado en disco.

6. Qué se aprendió del proceso

- Una petición de seguridad puede estar formulada sobre una premisa técnicamente imposible ("que no se pueda editar la URL"). Aceptarla al pie de la letra habría producido una falsa sensación de seguridad; lo correcto fue explicar por qué no se puede y atender el riesgo real que había detrás, que era entrada de usuario sin validar.
- Una interfaz puede verse perfectamente y ser inusable con teclado. El caso más revelador fue que el propio selector de tamaño de fuente una función pensada para accesibilidad no era alcanzable con Tab, y que un usuario de solo teclado no podía cerrar sesión.
- Los comentarios envejecen igual que el código, y sin la disciplina de revisarlos se convierten en documentación que miente: dos comentarios de este proyecto afirmaban lo contrario de lo que el código hacía, ambos por cambios de la propia sesión.
- Eliminar una función por un cambio de reglas de negocio puede arrastrar consigo un requisito obligatorio del proyecto que no tiene nada que ver con esa función. Conviene revisar los requisitos transversales antes de borrar una integración, no después.
- Una métrica automática necesita validarse antes de guiar decisiones: contar líneas de comentario sin distinguir prosa de anotaciones de tipo señalaba como problemáticos archivos que estaban perfectamente bien.
- Un rol con permisos asignados a mano en la base de datos, pero no en el seeder, es una bomba de tiempo: funciona hasta que alguien reconstruye el entorno desde cero.